StreamPulse

Real-Time Order Delivery Analytics & Data Engineering Platform
(Olist e-commerce dataset)

The Complete Project Guide

From absolute zero to a confident viva defence

Kafka · Spark Structured Streaming · Parquet · PostgreSQL · Frankfurter API · Airflow · Streamlit

Every number in this guide comes from a real run of this project.
The evidence files are listed in the appendix and in `docs/evidence_register.md`.

Honest framing: this is a *historical dataset replayed through a streaming pipeline*, not live orders.

Contents

How to use this guide

- Colour code used in the guide
- Suggested reading order

1. What is StreamPulse, in plain words?

- 1.1 The honest framing: what is real and what is not
- 1.2 The one-line journey
- 1.3 What we actually achieved (measured)

2. The vocabulary (read this before anything else)

- 2.1 About data and pipelines
- 2.2 About Kafka
- 2.3 About Spark and storage
- 2.4 About safety and correctness

3. The data: where it comes from and what it looks like

- 3.1 The source
- 3.2 The nine files (a small relational database in CSV form)
- 3.3 What we learned by profiling the data (before writing any code)
- 3.4 From many tables to one "order event"

4. The architecture: how everything fits together

- 4.1 Which technology does what, and why we chose it
- 4.2 The services and ports

5. Stage 1: The Python producer (from CSV rows to Kafka messages)

- 5.1 Our example order
- 5.2 The eight steps
- 5.3 The command-line switches

6. Stage 2: The real external API (Frankfurter)

- 6.1 What and why
- 6.2 The request
- 6.3 How the client protects us (src/api/client.py)
- 6.4 How a USD value is attached (src/api/enrich.py)

7. Stage 3: Kafka (the conveyor belt)

- 7.1 What Kafka does for us
- 7.2 How the topic is created and kept healthy
- 7.2b Why the key is order_id (a decision measured, not guessed)
- 7.3 Commands to look at Kafka

8. Stage 4: Spark Structured Streaming (the processing engine)

- 8.1 Why it runs in Docker
- 8.2 The big idea: process the stream as many small batches
- 8.3 The files
- 8.4 What happens inside one batch, step by step
- 8.5 The checkpoint (Spark's progress notebook)
- 8.6 Triggers and limits

8.7 Stopping safely (a real engineering story)
8.8 Commands for Spark

9. Stage 5: Data quality, duplicates and the dead-letter queue
9.1 Three promises
9.2 The rules (config/quality_rules.yaml, src/processing/validators.py)
9.3 What a dead-letter record looks like
9.4 The three kinds of "same record" (the most asked question)
9.5 A worked example: the Demo 3 test file
9.6 Parquet and PostgreSQL behave differently on purpose

10. Stage 6: PostgreSQL (the warehouse)
10.1 The structure
10.2 The key table: core.orders
10.3 The safe write (src/warehouse/load.py)
10.4 Reference loaders and run summaries
10.5 The analytics views (sql/analytics.sql)
10.6 The quality checks on the stored data

11. Stage 7: Airflow (running the steps as a workflow)
11.1 What Airflow is, and why it is here
11.2 How it is deployed
11.3 The DAG streampulse_batch: nine finite tasks
11.4 The real run
11.5 The retry and failure demonstration (streampulse_failure_demo)
11.6 Commands

12. Stage 8: The Streamlit dashboard (every page explained)
12.1 How the dashboard works
12.2 Page 1: Overview and status
12.3 Page 2: Dataset and provenance
12.4 Page 3: Ingestion and reconciliation
12.5 Page 4: Data quality
12.6 Page 5: Business KPIs
12.7 Page 6: External API
12.8 Page 7: Recent records
12.9 When services are really down

13. The folder and file map
14. The commands cookbook
14.1 First-time setup
14.2 Infrastructure
14.3 Ingestion
14.4 Spark, Parquet, PostgreSQL
14.5 Airflow, dashboard, tests

15. Reconciliation: proving the numbers add up
15.1 The idea
15.2 Three sections
15.3 Real output (shortened)
15.4 The "14 versus 15" story

16. Errors, retries, checkpoints and recovery

16.1 The protections, layer by layer

16.2 Retry, at-least-once and the idempotent sink

16.3 Checkpoint recovery: what exactly happens

16.4 The recovery experiment (isolated checkpoint)

16.5 Proof that re-ingestion is harmless

16.6 Dead-letter queue (DLQ) recap

16.7 The honest failures list

17. Testing

17.1 Why several suites

17.2 What is mocked and what is real

17.3 Status vocabulary

18. Limitations (say them before you are asked)

19. The four demonstrations

20. Viva questions and answers

Appendix A: Glossary

Appendix B: Key numbers (all measured)

Appendix C: Where to find the evidence

How to use this guide

This guide assumes you know **nothing** about data engineering. Read it once from the start. By the end you should be able to (1) run the project, (2) explain every part in your own words, and (3) answer a professor's questions honestly.

Colour code used in the guide

Box	Meaning
REAL OUTPUT green block	Text copied from a real run of this project (the source file is named).
COMMAND purple block	A PowerShell command you can type. Always run from the repository root.
CODE blue block	An excerpt from the project's source code.
Violet box	An **analogy**: a real-life comparison to make an idea easy.
Green box	**Why?** The reason a design choice was made.
Orange / red box	A warning or a real problem we met and fixed.

Suggested reading order

1. **Part 1** (chapters 1-4): the big picture, the vocabulary, the data, the architecture. Read this first, slowly.
2. **Part 2** (chapters 5-12): the journey of one order through every stage, then the dashboard.
3. **Part 3** (chapters 13-15): the folder map, every command and what its output means, reconciliation.
4. **Part 4** (chapters 16-20): errors and recovery, testing, limitations, the four demos, viva questions.
5. **Appendices**: full glossary, key numbers, evidence index.

> **If you only have one hour**
>
> Read chapter 1, chapter 2 (the words), the architecture picture in chapter 4, then chapters 9, 15, 16 (quality, reconciliation, recovery) and chapter 20 (viva). These are the parts a professor is most likely to ask about.

1. What is StreamPulse, in plain words?

An online shop (marketplace) receives orders all day. Each order has a customer, a status (*delivered*, *shipped*, *canceled*...), dates, money paid, and so on. A company wants to answer questions such as *How many orders per month? How much money did we make? How many deliveries were late?*

The raw order data is spread over several files and is messy. **StreamPulse is the machinery that takes the messy files, cleans and checks them, moves them step by step through a chain of tools, stores them safely, and finally shows the answers on a web dashboard.** Building that kind of machinery is called **data engineering**.

> **Analogy: a parcel factory**
> Imagine a factory that receives boxes (orders) from a warehouse. A worker opens each box and checks it (validation). Broken boxes go to a "problems" shelf with a note saying what is wrong (dead-letter queue). Good boxes are labelled and put on a conveyor belt (Kafka). Machines at the end of the belt (Spark) sort them into a big archive (Parquet) and into a neat catalogue where each box appears exactly once (PostgreSQL). A manager's screen (the dashboard) shows how many boxes there are. Auditors (reconciliation) count the boxes at every stage to be sure none got lost or counted twice.

1.1 The honest framing: what is real and what is not

Question	Honest answer
Is the data real?	**Yes.** It is the public *Brazilian E-Commerce Public Dataset by Olist* (about 100,000 real, anonymised orders from 2016-2018).
Is it food delivery?	**No.** It is e-commerce (many product types). The original project title said "Food Delivery"; we changed the subtitle because the dataset is not food. *You should confirm this with your professor.*
Are the orders live?	**No.** They are historical. We **replay** them through a streaming pipeline at a speed we control. Always say: "*historical dataset replay through a streaming pipeline*".
Is the API real?	**Yes.** The Frankfurter API (European Central Bank exchange rates) is a real public service. It does **not** contain orders; it gives currency rates.
Is it "exactly-once"?	**No.** It is *at-least-once delivery with an idempotent (safe-to-repeat) database write*. Chapter 16 explains the difference.

1.2 The one-line journey

```
Olist CSV files  ->  Python producer  ->  Kafka  ->  Spark  ->  Parquet files + PostgreSQL  ->  Streamlit dashboard
                     (check, join)     (conveyor)  (process)   (archive)   (clean catalogue)      (see the answers)
                                  + Frankfurter API adds a USD value to every order
                                  + Airflow runs the repeatable steps as a scheduled workflow
```

1.3 What we actually achieved (measured)

Result	Value	Where it is proved
Orders in the real file	99,441	`docs/evidence/01_dataset/dataset_profile.txt`
Orders accepted (valid)	99,427	`03_kafka/producer_full_run.txt`
Orders rejected (contradict themselves)	14 (kept in the dead-letter queue)	`05_data_quality/` , `07_postgresql/rejected_14_vs_15_explained.txt`
Rows in the warehouse	99,427 (each order exactly once)	`07_postgresql/quality_checks_output.txt`
Total revenue in the data	16,006,744.62 BRL (about 4.77 million USD, estimated)	`07_postgresql/analytics_views_output.txt`
Late deliveries	8.11 %	same file
Automated tests	73 unit + 18 integration + 5 Spark + 8 Airflow + 4 end-to-end, all passing	`10_automated_tests/final_*.txt`

2. The vocabulary (read this before anything else)

Most confusion in data engineering comes from unfamiliar words. Here are the essential ones, each with a plain explanation. A longer glossary is at the end of the guide.

2.1 About data and pipelines

Word	Plain meaning	Example in this project
Data pipeline	A chain of steps that moves data from where it starts to where it is useful.	CSV → Python → Kafka → Spark → PostgreSQL → dashboard
Batch vs **stream**	Batch = process a big pile at once, then stop. Stream = process small pieces continuously as they arrive.	Reading the 100,000-order file is a pile; Kafka + Spark treat it as a flow of messages.
Event / message	One small piece of information that something happened.	"Order X was placed at 21:15 for 136.23 BRL" is one message.
Replay	Sending old recorded data through a system as if it were new.	We replay 2016-2018 orders in a few minutes.
ETL / ELT	Extract-Transform-Load (clean first, then store) vs Extract-Load-Transform (store first, clean later).	Mostly ETL: we check and transform before loading; SQL views do the final analytics.
Validation	Checking that data follows the rules.	"A delivered order must have a delivery date."
Enrichment	Adding extra information from another source.	Adding the USD value from the exchange-rate API.
Provenance / lineage	Knowing where every piece of data came from.	Each row remembers its source file hash, row number, run id, Kafka position.

2.2 About Kafka

Word	Plain meaning
Kafka	A system that stores and delivers messages in order, like a very reliable conveyor belt that remembers everything for a few days.
Broker	The Kafka server (ours runs in a Docker container).
Topic	A named channel of messages. Ours is `ecommerce_orders`.
Partition	A topic is split into several lanes (ours: 3) so work can be shared. Order is guaranteed only *inside* one lane.
Offset	The position number of a message inside a lane (0, 1, 2, ...). Like a page number.
Key	A label on a message that decides which lane it goes to. Same key → same lane. Ours: the `order_id`.
Producer	A program that *puts* messages into Kafka (our Python replay script).
Consumer	A program that *reads* messages from Kafka (Spark).

2.3 About Spark and storage

Word	Plain meaning
Spark Structured Streaming	A tool that reads a flow of data in small groups and processes each group like a mini batch.
Micro-batch	One small group of messages processed together (ours: up to 10,000 messages).
Trigger	The rule for *when* to run the next micro-batch: every 5 seconds, or `availableNow` = "process everything waiting, then finish".
Checkpoint	Spark's progress notebook: which Kafka offsets have been processed. After a stop, Spark continues from there.
Parquet	A compact file format for tables, good for analytics. Think "a very efficient Excel file".
Landing layer	The first place data is stored after processing, kept as-is for audit. Ours: Parquet files.
Data warehouse	A clean database prepared for questions and reports. Ours: PostgreSQL.
View	A saved SQL question that looks like a table (e.g. "orders per month").
KPI	Key performance indicator: an important number such as revenue or late-delivery rate.

2.4 About safety and correctness

Word	Plain meaning
Idempotent	Doing it twice has the same result as doing it once. Pressing a lift button twice does not call two lifts.
Upsert	"Update or insert": add the row if it is new, skip (or update) it if it already exists.
Duplicate	The same thing appearing more than once. We distinguish *retry duplicates* and *source duplicates* (chapter 9).
Dead-letter queue (DLQ)	A "problems shelf": rejected records are stored with the reason instead of being thrown away.
At-least-once	Every message is delivered one *or more* times; none is lost, but repeats are possible.
Exactly-once	Every message is processed exactly one time. Hard to guarantee across many systems; **we do not claim it**.
Reconciliation	Counting records at every stage and checking that the numbers agree (or are explained).
Orchestrator / DAG	A tool (Airflow) that runs steps in the right order with retries. A DAG is the "recipe": tasks and arrows, no loops.
Container / Docker	A sealed, ready-to-run package of a program and everything it needs. Our Kafka, PostgreSQL, Spark and Airflow run in containers.

3. The data: where it comes from and what it looks like

3.1 The source

Item	Value
Name	**Brazilian E-Commerce Public Dataset by Olist**
Where	Kaggle: `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`
Owner	Olist (a Brazilian marketplace). The data is real but **anonymised**.
Period	September 2016 - October 2018 (first order 2016-09-04 21:15:19, last 2018-10-17 17:30:18)
License	CC BY-NC-SA 4.0: you must credit Olist, use it non-commercially, and share alike. Fine for coursework. *(Read from the Kaggle page via a search excerpt: re-confirm in a browser.)*
Where it lives in the repo	`data/raw/` (git-ignored: the files are never committed; see `data/raw/README.md`)
Rule	The raw files are **never edited**. The project only reads them. A script (`scripts/check_dataset_fingerprint.py`) proves they are unchanged using SHA-256 fingerprints.

> **Analogy: a fingerprint**
> A SHA-256 hash is a fingerprint of a file. If even one character changes, the fingerprint changes completely. We recorded the fingerprints on day one, so we can prove later that the data was not modified.

3.2 The nine files (a small relational database in CSV form)

Real shops do not keep everything in one table. The data is split into tables that point to each other with **keys** (ID columns), like a set of index cards that refer to each other.

File	Rows	Cols	What one row is	Role in StreamPulse
`olist_orders_dataset.csv`	99,441	8	one order	**the main thing we stream**
`olist_order_items_dataset.csv`	112,650	7	one item inside an order	summed per order; also loaded as reference table
`olist_order_payments_dataset.csv`	103,886	5	one payment of an order	summed per order (revenue)
`olist_customers_dataset.csv`	99,441	5	one customer record (per order)	city and state of the order
`olist_order_reviews_dataset.csv`	99,224	7	one review	reference table (review scores)
`olist_products_dataset.csv`	32,951	9	one product	reference table (categories)
`olist_sellers_dataset.csv`	3,095	4	one seller	reference table

`product_category_name_translation.csv`	71	2	Portuguese to English category name	translates product categories
`olist_geolocation_dataset.csv`	1,000,163	5	one latitude/longitude sample	reduced to one point per zip prefix (19,010)

The orders file, column by column

Column	Meaning	Empty values
`order_id`	unique ID of the order (a long hex text). **No duplicates in the file** (verified).	0
`customer_id`	key that points into the customers file	0
`order_status`	one of 8 values (below)	0
`order_purchase_timestamp`	when the order was placed	0
`order_approved_at`	when payment was approved	160
`order_delivered_carrier_date`	when it was handed to the courier	1,783
`order_delivered_customer_date`	when the customer received it	2,965
`order_estimated_delivery_date`	the delivery date promised to the customer	0

Important idea: an *empty* delivery date is **not an error**. A canceled or still-shipping order simply has no delivery date yet. Our rules never reject an order only because a date is empty (they reject only *contradictions*, chapter 9).

The eight order statuses (real counts)

delivered	shipped	canceled	unavailable	invoiced	processing	created	approved
96,478	1,107	625	609	314	301	5	2

3.3 What we learned by profiling the data (before writing any code)

Profiling means measuring the data to learn its real shape. We did this with `scripts/profile_dataset.py` and saved the report. We never assumed column names or counts.

Finding	Number	What we decided
Duplicate `order_id` in the orders file	0	`order_id` is a safe *natural key* (a real-world unique ID).
Orders with no item rows	775 (603 unavailable, 164 canceled, ...)	Keep them with `items_count = 0`; never silently drop.
Orders with no payment row	1	Keep with `payments_count = 0`.
Delivered orders without a delivery date	8	Contradiction → reject to the dead-letter queue.
Orders with a delivery date but status not "delivered"	6 (all canceled)	Contradiction → reject.
Delivered before purchase	0	Rule kept anyway (used in Demo 3).

Delivered after the promised date (late)	7,827	Not invalid! This is the *late-delivery KPI*.
`customer_unique_id` repeating across `customer_id`	3,345	Normal: one real person can place several orders.
Reviews with a repeated `review_id`	814 rows	Reference table key is (review_id, order_id).
Geolocation exact duplicate rows	261,831 of 1,000,163	Reduce to one centroid per zip prefix.
Geolocation points outside Brazil	42	Ignored for centroids and reported.
Products without a category / category names without translation	610 / 2 names	Show the Portuguese name when no translation exists.

3.4 From many tables to one "order event"

A message in Kafka should describe **one order completely**. So before sending, Python **joins** (combines) the tables. This is implemented in `src/common/orders_source.py`:

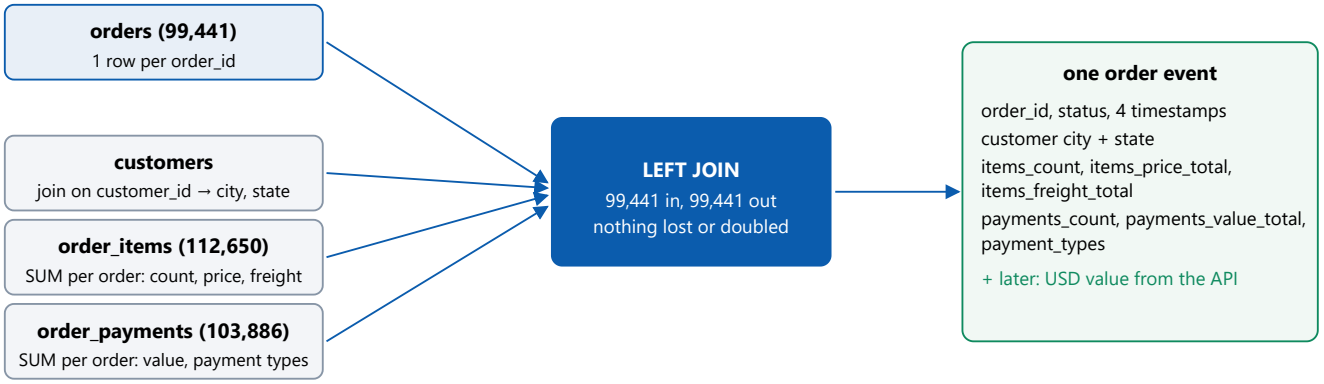


Figure 3.1: Four tables become one self-contained order event. LEFT JOIN keeps orders even if a child table has no rows for them.

> **Why LEFT JOIN?**
>
> A normal join would drop the 775 orders that have no item rows. A **LEFT JOIN** keeps every order and fills missing parts with 0. Rule 8 of our project: *never silently drop rows*. A check script proves the result: `99,441 orders in, 99,441 order rows out` (`docs/evidence/01_dataset/join_and_mapping_check.txt`).

Replay order. The rows are sorted by `order_purchase_timestamp`, so the replay follows real history from September 2016 to October 2018.

4. The architecture: how everything fits together

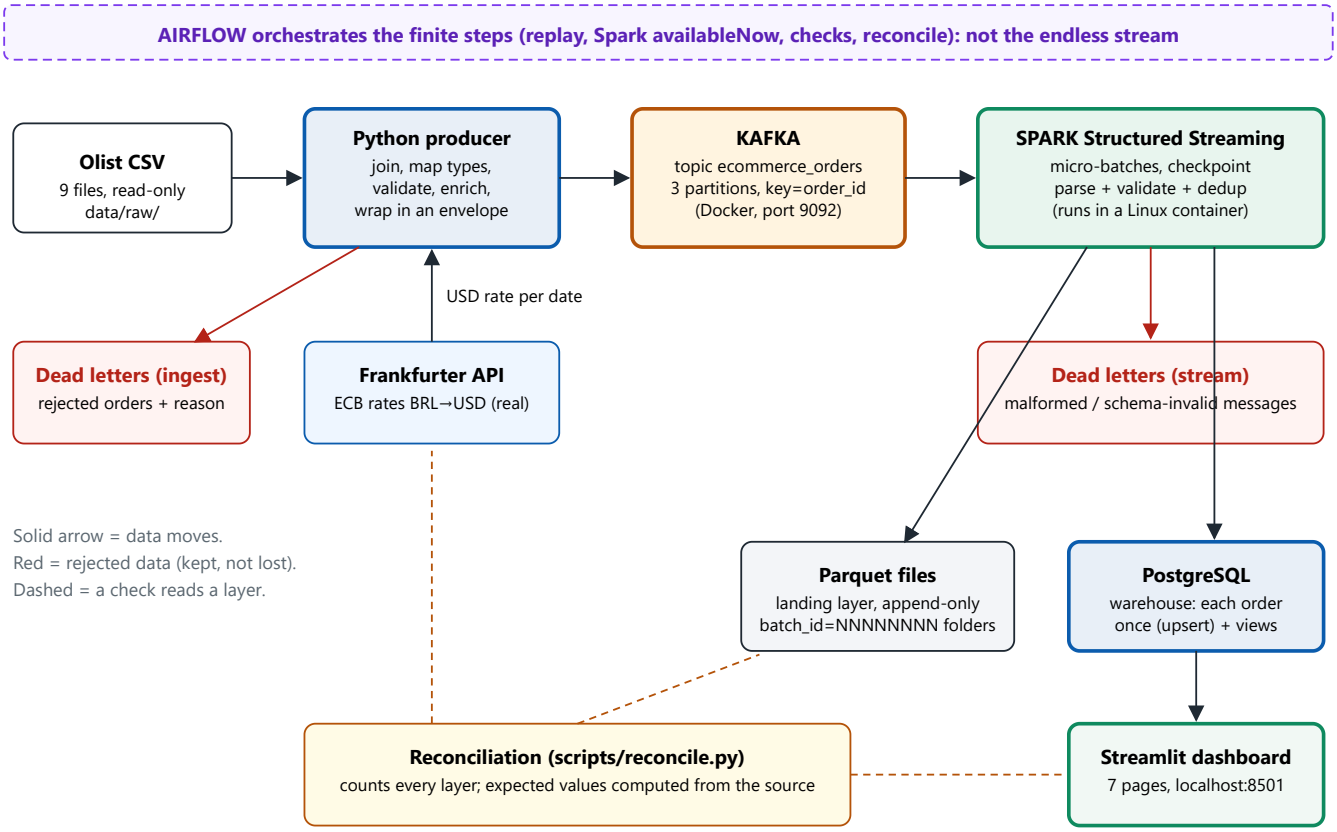


Figure 4.1: The StreamPulse architecture. Only connections that exist in the code and were run are drawn.

4.1 Which technology does what, and why we chose it

Tool	Its job here	Why this choice (and what we did not choose)
Python 3.11	Producer, API client, validation rules, loaders, scripts, tests, dashboard code.	Readable for beginners; huge ecosystem (pandas, kafka-python, psycopg2, Streamlit).
Apache Kafka 4.1	Durable message conveyor between the producer and Spark. Remembers messages (offsets) so a restarted consumer can continue.	It is the standard tool for streaming data and the one the professor expects. It decouples fast producers from slower consumers.
Spark 4.2 Structured Streaming	Reads Kafka in micro-batches, validates, removes in-batch duplicates, writes Parquet and PostgreSQL.	Handles growth (the same code could run on a cluster) and gives checkpoints for recovery. Runs in Docker because on Windows it needs `winutils.exe` (an untrusted third-party download we refused).
Parquet	Landing layer: every valid message Spark consumed, kept for audit and replay.	Compact, columnar, typed. Simple files, no server.

PostgreSQL 16	The warehouse: one clean row per order, plus reference tables and analytics views.	Free, reliable, great SQL. `INSERT ... ON CONFLICT` gives the safe upsert.
Frankfurter API	Real external data: historical BRL→USD exchange rates.	No key, no quota, official ECB rates, **one** request covers the whole period. (Weather API needs thousands of calls; a state-name API adds little.)
Airflow 2.10	Runs the repeatable steps in order, with retries and logs.	The standard orchestrator. Used only for *finite* tasks: an endless stream is not a task.
Streamlit	The dashboard.	A dashboard in pure Python, no web-development skills needed.
Docker Compose	Starts Kafka, PostgreSQL, Spark and Airflow the same way on any machine.	Reproducible setup, nothing to install by hand.

4.2 The services and ports

Service	Container name	Port (your PC)	Started by
Kafka broker	`streampulse-kafka`	9092 (containers use `kafka:29092`)	`scripts\kafka_start.ps1`
PostgreSQL	`streampulse-postgres`	5432	`scripts\init_db.ps1`
Spark job	a temporary container per run	none	`scripts\run_stream.ps1`
Airflow	`streampulse-airflow`	8080	`scripts\airflow_up.ps1`
Dashboard	(runs directly on Windows)	8501	`scripts\run_dashboard.ps1`

> **Why two Kafka addresses?**
> Inside a container, "localhost" means *the container itself*. So Kafka advertises two doors: `localhost:9092` for programs on your PC (the producer) and `kafka:29092` for other containers (Spark, Airflow). This is configured in `docker-compose.yml`.

5. Stage 1: The Python producer (from CSV rows to Kafka messages)

File: `src/producer/dataset_producer.py`. This is the program you run to start the flow. It does eight things for every order. Follow one real order through them.

5.1 Our example order

The very first order in replay order (it was the first message Kafka stored in partition 0). Real values from `docs/evidence/03_kafka/consume_sample_messages.txt`:

Field	Value
order_id	`2e7a8482f6fb09756ca50c10d7bfc047`
status	shipped
purchase / approved / carrier	2016-09-04 21:15:19 / 2016-10-07 13:18:03 / 2016-10-18 13:14:51
delivered to customer	(empty: not delivered yet, perfectly valid)
customer	boa vista, state RR
items	2 items, total price 72.89, freight 63.34
payment	1 payment by credit card, total 136.23 BRL

5.2 The eight steps

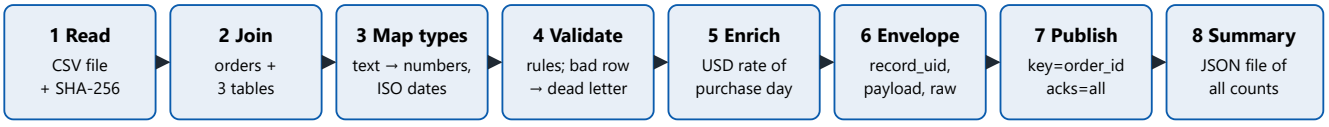


Figure 5.1: What the producer does to every order.

Step 1: read and fingerprint

The orders file is opened and its SHA-256 fingerprint is computed. That fingerprint is used to create the record's identity (step 6).

Step 2: join

Explained in chapter 3 (`build_order_rows`). Result: one row per order with customer city/state and the item/payment totals.

Step 3: map and convert types (`config/field_mapping.yaml`, `src/common/mapping.py`)

CSV values are all *text*. A **mapping file** says, for every field: which source column it comes from, its new name, its type, and whether it is required. Excerpt:

CONFIG/FIELD_MAPPING.YAML

```
  - {source_column: order_purchase_timestamp,      canonical_name: purchase_ts,           type: timestamp,
required: true,  origin: orders}
  - {source_column: order_delivered_customer_date, canonical_name: delivered_customer_ts, type: timestamp,
required: false, origin: orders}
  - {source_column: items_count,                   canonical_name: items_count,           type: int,
required: true,  origin: order_items}
```

`apply_mapping()` converts each value (text → number, text → ISO timestamp such as `2016-09-04T21:15:19`) and returns **either a clean record or a list of errors**. It never crashes on bad data; it reports it. Error codes: `MISSING_REQUIRED` (a required value is empty) and `TYPE_ERROR` (e.g. "abc" where a number is needed).

> **Why a mapping file and not hard-coded names?**
>
> Rule 7 of the project: *never assume column names*. The mapping file is built from the real profiled columns, so it can be read, reviewed, and changed without touching code. Spark even builds its message schema from the same file, so producer and consumer can never disagree.

Step 4: validate (`config/quality_rules.yaml`, `src/processing/validators.py`)

Nine rules check the order (chapter 9 explains each). If any *reject* rule fires, the order goes to the dead-letter file with the reason and is **not** published. The single function `classify_record()` decides this; the reconciliation script calls the same function, so the two can never disagree.

Step 5: enrich with the API

Chapter 6. Before the loop starts, the producer asks the API **once** for the BRL→USD rates of the whole date range, then looks up each order's purchase date in that table. If the API is down the producer logs a warning and continues without enrichment (it never stops the ingestion).

Step 6: build the envelope (the message)

A Kafka message is a standard "envelope" around the order. Real message consumed from Kafka (shortened with "..." only where long):

REAL OUTPUT · DOCS/EVIDENCE/03_KAFKA/CONSUME_SAMPLE_MESSAGES.TXT


```
{
  "schema_version": "1.0",
  "record_uid": "fc6fdc70c71fd67c6611ac15899fad76",
  "business_hash": "35d18ab7f8edeb353e6f06b79181cbfb",
  "source_category": "kafka_replayed",
  "source_file": "olist_orders_dataset.csv",
  "source_file_sha256": "8df58ef3d2d7e9944010f7beecd9b75367f5588ec6e3c91cec19ae3345ef9ecf",
  "source_row_number": 4542,
  "ingest_run_id": "limited-100",
  "ingested_at": "2026-10-15T16:38:02.702715+00:00",
  "payload": { "order_id": "2e7a8482...c047", "order_status": "shipped",
               "purchase_ts": "2016-09-04T21:15:19", "delivered_customer_ts": null,
               "customer_city": "boa vista", "customer_state": "RR",
               "items_count": 2, "items_price_total": 72.89, "items_freight_total": 63.34,
               "payments_count": 1, "payments_value_total": 136.23, "payment_types": "credit_card", ... },
  "raw": { "order_id": "...", "order_purchase_timestamp": "2016-09-04 21:15:19", ... original source
columns ... },
  "api_enrichment": { "fx_rate_brl_usd": 0.30625, "fx_rate_date": "2016-09-02",
                      "fx_match_type": "previous_business_day", "payments_value_usd": 41.72,
                      "api_source_name": "frankfurter-ecb", "api_http_status": 200, ... }
}
```

Field	Meaning
`record_uid`	The **identity of this record**: a hash of (file fingerprint + row number). Same file + same row ⇒ same ID, always. This is what makes repeats recognisable.
`business_hash`	A hash of the natural key `order_id`. Two different rows with the same business hash are about the *same order* (a source duplicate).
`source_category`	`kafka_replayed`: honest label that this was replayed, not live.
`source_file_sha256`, `source_row_number`	Provenance: exactly which file and row it came from.
`ingest_run_id`	Which execution of the producer sent it (e.g. `limited-100`, `full-run-01`).
`payload`	The clean, typed order (what the warehouse stores).
`raw`	The original source values, preserved (never lose the original).
`api_enrichment`	The USD value and the API provenance (endpoint, time, HTTP status).

SRC/PRODUCER/DATASET_PRODUCER.PY

```
def make_record_uid(file_sha256_value: str, row_number: int) -> str:
    """Same file + same row => same ID, so a re-run can be recognised downstream."""
    return hashlib.sha256(f"{file_sha256_value}:{row_number}".encode()).hexdigest()[:32]

def make_business_hash(order_id: str) -> str:
    """order_id is a verified natural key (no duplicates in the real orders file)."""
    return hashlib.sha256(f"order_id:{order_id}".encode()).hexdigest()[:32]
```

Analogy: two kinds of ID

`record_uid` is like the *serial number of one photocopy* (this file, this row). `business_hash` is like the *title of the document*. Two photocopies of the same page have different serial numbers (different record_uid) but the same title (same business_hash).

Step 7: publish to Kafka

PRODUCER SETTINGS (CREATE_PRODUCER)

```
acks="all"               # wait until the broker confirms the write is safely stored
enable_idempotence=True  # the broker ignores duplicates caused by the producer's own retries
retries=5, retry_backoff_ms=200
linger_ms=20, batch_size=64*1024, compression_type="gzip"   # send in efficient batches
key = order_id           # decides the partition
headers = ingest_run_id, schema_version
```

Each send returns a "future". Two small callbacks count **successes** (and which partition got the message) and **failures**. Every 500 valid records the producer calls `flush()` and logs progress. `Ctrl+C` is handled: it flushes what was already sent and still writes the summary.

Step 8: the run summary

At the end a JSON file is written to `data/processed/ingestion_runs/<run_id>.json`. This is the proof that nothing was lost. The real full run:

REAL OUTPUT · DOCS/EVIDENCE/03_KAFKA/PRODUCER_FULL_RUN.TXT


```
"status": "completed",   "mode": "full",
"rows_read": 99441,      "rows_valid": 99427,     "rows_rejected": 14,
"rows_published": 99427, "rows_publish_failed": 0,
"rows_enriched": 99427,  "rows_not_enriched": 0,  "source_duplicates_in_input": 0,
"partition_counts_from_delivery_callbacks": { "0": 33199, "1": 33255, "2": 32973 }
```


How to read it: 99,441 rows were read; 99,427 passed the checks; 14 were rejected (and stored); all 99,427 were delivered to Kafka with zero failures; every one received a USD value. The three partition numbers add up to 99,427. This took about 100 seconds.

5.3 The command-line switches

Switch	Meaning
`--limit N`	Send only the first N orders (in replay order).
`--full`	Send everything. **Required on purpose**: the program refuses to start unless you pick `--limit` or `--full`, so a 100,000-message run can never start by accident.
`--dry-run`	Do all checks and counting but publish nothing.
`--skip N`	Skip the first N orders (used to make non-overlapping test slices).
`--run-id NAME`	Give this execution a name (used in logs, files and the database).
`--rate R`	Records per second (0 = as fast as possible).
`--input FILE`, `--topic T`, `--bootstrap HOST`	Use another orders-shaped CSV, another topic, another broker (used for demos and tests).
`--no-enrich`	Skip the API step.
`--key-field F`	Which field is the Kafka key (default `order_id`).

6. Stage 2: The real external API (Frankfurter)

6.1 What and why

Every order has a **money value in Brazilian reais (BRL)** and a **purchase date**. The **Frankfurter** service publishes official European Central Bank exchange rates for past dates. So we can add: "*what was this order worth in US dollars on the day it was bought?*"

> **Why this API?**
> The professor did not name an API (the file `docs/inputs/api_info.md` does not exist), so we chose one that matches the real columns. Frankfurter needs **no key**, has **no quota**, and **one request** returns about 544 daily rates for the whole 2016-2018 period. A weather API would have needed thousands of calls. Important honesty: **public APIs do not contain Olist orders.** This API provides reference data, and that is its role.

6.2 The request

COMMAND

```
$py scripts\api_demo_call.py --sample --no-cache
```

REAL OUTPUT · DOCS/EVIDENCE/02_API/API_REAL_REQUEST.TXT

```
Provider : Frankfurter (ECB reference rates), docs https://frankfurter.dev
Request  : BRL -> USD, 2017-10-02 to 2017-10-08
Retrieved: 2026-10-15T16:30:20.762754+00:00
URL      : https://api.frankfurter.dev/v2/providers/ecb/rates?base=BRL&quotes=USD&from=2017-10-02&to=2017-
10-08
HTTP     : 200   (served from local cache: False)
Rows     : 5
    {"date": "2017-10-02", "base": "BRL", "quote": "USD", "rate": 0.31507}
Enrichment on sample purchase dates:
   2017-10-07 -> (0.31641, '2017-10-06', 'previous_business_day')
```

Reading it: HTTP **200** means "OK, here is your data". 1 BRL was worth 0.31507 USD on 2 October 2017. 2017-10-07 was a Saturday and the ECB publishes no weekend rates, so the program used Friday's rate and labelled it `previous_business_day`.

6.3 How the client protects us (`src/api/client.py`)

Feature	What it does	Why
Timeouts	5 s to connect, 30 s to read.	Never wait forever.
Retry with exponential backoff	Up to 4 retries on timeouts, network errors, HTTP 429 and 5xx, waiting 1 s, 2 s, 4 s...; obeys a `Retry-After` header.	Temporary problems fix themselves; a client must be polite when the server is busy.
No retry on other 4xx	A "bad request" (e.g. unknown currency) fails immediately.	Retrying cannot fix it.
Throttle	At least 0.5 s between calls.	Respect the provider's rate limit.

Schema validation	The answer must be a non-empty list of `{date, base, quote, rate}` with a positive number.	Never trust an external answer blindly.
File cache	The answer is saved in `data/processed/api_cache/`, keyed by the exact request. The same range is never requested twice.	Speed, and being kind to the provider (they encourage caching).
Provenance fields	Endpoint (secrets masked), retrieval time, HTTP status, cache key are attached to every result.	Traceability: you can always say where a USD value came from.

The retry behaviour was shown against a host that does not exist (a real client, an unreachable server; *not* a fake API):

`REAL OUTPUT · DOCS/EVIDENCE/02_API/API_RETRY_BACKOFF_SIMULATED.TXT`

```
WARNING API attempt 1 failed: ConnectionError
WARNING Retrying in 1.0s (attempt 1 of 2)
WARNING API attempt 2 failed: ConnectionError
WARNING Retrying in 2.0s (attempt 2 of 2)
WARNING API attempt 3 failed: ConnectionError
RESULT   : request failed after retries: API failed after 3 attempts: ConnectionError
```

6.4 How a USD value is attached (`src/api/enrich.py`)

1. The lookup key is the purchase **date** (about 600 different dates), never one call per order. Rule 10: *do not call an API 99,441 times*.
2. Exact date found ⇒ `fx_match_type = exact`.
3. Weekend or holiday ⇒ use the latest earlier rate (at most 7 days back) ⇒ `previous_business_day`.
4. No rate found ⇒ `api_enrichment = null` (counted, never hidden).
5. `payments_value_usd = payments_value_total × rate`, rounded to cents.

Our example order was bought on a Sunday (2016-09-04), so it used the rate of Friday 2016-09-02 (0.30625): 136.23 BRL × 0.30625 = **41.72 USD**.

Measured result	Value
Requests needed for the whole dataset	1 (HTTP 200, 544 daily rates, 2016-09-04 to 2018-10-17)
Orders that got a USD value	99,427 of 99,427 (match rate 100 %)
Orders that used the previous business day	24,112 (weekends/holidays)
Where the rates are stored	PostgreSQL `core.api_cache` (raw response) and `core.fx_rates` (one row per date)

> **Be honest about the USD numbers**
> They are **estimates**: BRL value × the ECB reference rate of the purchase day. Customers actually paid in BRL.

7. Stage 3: Kafka (the conveyor belt)

7.1 What Kafka does for us

Without Kafka, the producer would have to hand data directly to Spark, and both would need to run at the same speed at the same time. Kafka sits in the middle and **stores** the messages. The producer can finish and go away; Spark can start later, stop, restart and continue from where it left off, because Kafka remembers every message and its position (offset).

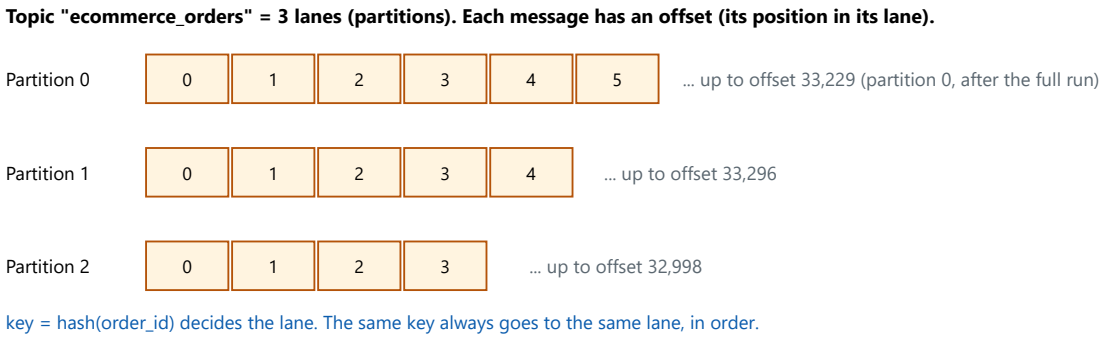


Figure 7.1: One topic, three partitions, each message numbered by offset. (Counts are the real offsets after the full run.)

7.2 How the topic is created and kept healthy

- `docker-compose.yml` starts Kafka (image `apache/kafka:4.1.0`, "KRaft" mode: no extra ZooKeeper server).
- A **healthcheck** asks the broker for its API versions every 10 seconds; "healthy" means it answers.
- A one-shot service `kafka-init` then creates the topic explicitly: `--partitions 3 --replication-factor 1`.
- Replication factor 1 = one copy of each message (we have one broker). In production you would use at least 3.
- Kafka keeps messages for about 7 days by default. After that you would simply replay again (safe, because replays are idempotent).

7.2b Why the key is `order_id` (a decision measured, not guessed)

The key decides how evenly work is shared. Before publishing we computed (offline, using Kafka's own hash formula) how three candidate keys would spread the real orders:

Key	Lane 0	Lane 1	Lane 2	Verdict
`order_id` (99,441 different values)	33.4 %	33.4 %	33.2 %	**Even. Chosen.**
`customer_id`	33.2 %	33.4 %	33.3 %	Even too, but no benefit.
`customer_state` (only 27 values)	22.2 %	**66.9 %**	10.9 %	Badly skewed (São Paulo alone is 42 % of orders).

Then, after the real run, we **measured on the broker** (not just on paper):

`REAL OUTPUT · DOCS/EVIDENCE/03_KAFKA/PARTITION_COUNTS_AFTER_FULL_RUN.TXT`

```
ecommerce_orders  partition 0: start_offset=0 end_offset=33230 messages=33230
ecommerce_orders  partition 1: start_offset=0 end_offset=33297 messages=33297
ecommerce_orders  partition 2: start_offset=0 end_offset=32999 messages=32999
```

(These include the 99 messages of an earlier 100-order test run; the full run itself added 33,199 + 33,255 + 32,973 = 99,427, exactly what the producer's delivery callbacks reported.)

> **Why does order matter inside one lane?**
> Kafka only promises order *within* a partition. Each order is an independent event, so we do not need orders of *different* customers in sequence. We just need an even spread, so `order_id` is perfect. If we had needed "all events of one customer in order", we would have keyed by customer.

7.3 Commands to look at Kafka

COMMANDS

```
.\scripts\kafka_describe_topic.ps1                # shape of the topic
.\scripts\kafka_partition_counts.ps1              # MEASURED messages per partition
.\scripts\kafka_consume_n.ps1 -Count 1 -Partition 0   # read one real message
```

REAL OUTPUT · KAFKA_DESCRIBE_TOPIC.PS1

```
Topic: ecommerce_orders  PartitionCount: 3  ReplicationFactor: 1  Configs: min.insync.replicas=1
  Partition: 0  Leader: 1  Replicas: 1  Isr: 1
  Partition: 1  Leader: 1  Replicas: 1  Isr: 1
  Partition: 2  Leader: 1  Replicas: 1  Isr: 1
```

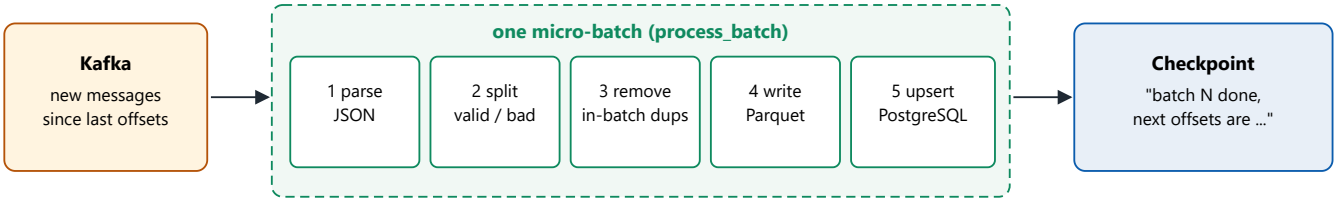
Meaning: three lanes; the "leader" of each is broker number 1 (our only broker); "Isr" = in-sync replicas (the copies that are up to date).

8. Stage 4: Spark Structured Streaming (the processing engine)

8.1 Why it runs in Docker

On Windows, Spark needs a helper program `winutils.exe` to write files. When we first ran Spark on the host it crashed at start-up: `HADOOP_HOME and hadoop.home.dir are unset` (real log: `docs/evidence/00_environment/spark_baseline_existing_stream.txt`). The only fix is downloading an unofficial executable, which our rules forbid. So Spark runs in a small **Linux container** (`docker/spark/Dockerfile`), reading Kafka through its internal address and writing to the shared `data/` folder. Bonus: it works identically on any computer.

8.2 The big idea: process the stream as many small batches



repeat: every trigger (e.g. 5 seconds) or until no data is left (availableNow). Up to 10,000 messages per micro-batch.

Figure 8.1: One micro-batch. Only after all steps finish is the batch "committed" in the checkpoint.

8.3 The files

File	Role
`src/streaming/order_stream.py`	The job: reads Kafka, calls `process_batch` for each micro-batch, handles stopping.
`src/streaming/transforms.py`	Pure DataFrame functions (parse, split, flatten, dedup). Pure = easy to test on small tables.
`scripts/run_stream.ps1`, `stop_stream.ps1`	Start and safely stop the container job.

8.4 What happens inside one batch, step by step

1. **Parse.** The Kafka message value is bytes. It is converted to text and parsed with `from_json` using a schema **generated from the same mapping file** the producer used. The Kafka position (partition, offset) is kept.
2. **Split valid / invalid.** A message is invalid if it is not valid JSON (`MALFORMED_JSON`) or lacks `record_uid` / `order_id` (`SCHEMA_MISMATCH`). Invalid ones are written to `data/dead_letter/stream/` with the original text. (They do **not** crash the job.)
3. **Flatten.** The nested envelope becomes one flat row: envelope fields, typed order fields (timestamps become real timestamps), USD enrichment, original source columns as JSON text, Kafka partition and

offset.

4. **Remove duplicates inside the batch.** If the same `record_uid` appears twice in this batch, keep the one with the smallest Kafka position.
5. **Write Parquet.** Into its own folder `data/processed/parquet/batch_id=00000007/`. If the same batch is ever run again, it overwrites **its own folder only**.
6. **Upsert into PostgreSQL** (chapter 10) inside one database transaction.
7. **Log counts.** One summary line is logged and appended to `data/processed/spark_batches/orders.jsonl`.

Spark then writes "batch done" into the checkpoint. Real summary of the very first production batch:

REAL OUTPUT · DATA/PROCESSED/SPARK_BATCHES/ORDERS.JSONL (FIRST LINE)


```
{"batch_id": 0, "input_rows": 9998, "valid_rows": 9998, "invalid_rows": 0, "in_batch_duplicates": 99,
 "parquet_rows_written": 9899,
 "postgres": {"attempted": 9899, "inserted": 9899, "retry_duplicates": 0, "source_duplicates": 0,
"batch_id": 0},
 "offsets": [{"partition": 0, "first_offset": 0, "last_offset": 3337},
             {"partition": 1, "first_offset": 0, "last_offset": 3344},
             {"partition": 2, "first_offset": 0, "last_offset": 3314}], "seconds": 24.27}
```


Field	Meaning (for this real batch)
`input_rows: 9998`	Spark read 9,998 messages from Kafka (the limit per batch is 10,000).
`valid_rows: 9998, invalid_rows: 0`	All parseable and complete.
`in_batch_duplicates: 99`	99 messages repeated a `record_uid` already in this batch (the first 99 orders had been sent twice: once in a 100-order test run, once in the full run).
`parquet_rows_written: 9899`	9,998 − 99 = 9,899 rows written.
`postgres.inserted: 9899`	All 9,899 were new to the warehouse. (`retry_duplicates` and `source_duplicates` were 0.)
`offsets`	Which Kafka positions this batch covered. The next batch starts right after these.
`seconds: 24.27`	The first batch is slow (Spark warm-up); later ones took 5-8 seconds.

All 10 batches of the first full pass processed **99,526 messages** (99,427 unique orders plus 99 repeats).

8.5 The checkpoint (Spark's progress notebook)

REAL FOLDER · DATA/CHECKPOINTS/ORDERS (NEVER DELETE THIS BY HAND)

```
commits/   0 1 2 3 ... 13     one tiny file per FINISHED batch
offsets/   0 1 2 3 ... 13     which Kafka offsets each batch was PLANNED to read
metadata                      the id of this streaming query
sources/                      bookkeeping for the Kafka source

offsets/0  last line:  {"ecommerce_orders":{"0":3338,"1":3345,"2":3315}}
```

How to read it: after batch 0 the notebook says "next time start reading partition 0 at offset 3338, partition 1 at 3345, partition 2 at 3315". If Spark is stopped and started again with the same checkpoint, it reads this and continues exactly there. A batch has an `offsets` file when it is *planned* and a `commits` file when it is *finished*. If

a batch is planned but not committed (a crash), it is simply **run again** after the restart. That is safe only because our writes are idempotent.

> **Analogy: a bookmark**
> The checkpoint is a bookmark in a long book (Kafka). If you close the book (stop Spark), you do not start from page 1 again: you open it at the bookmark.

> **Never delete the checkpoint to "fix" something**
> Without it, Spark would re-read the topic from the beginning (we set `startingOffsets=earliest` for the first run only). Our scripts that clean up demos are restricted to demo folders and can never touch `data/checkpoints/orders`.

8.6 Triggers and limits

Setting	Meaning
`--trigger "5 seconds"` (default)	Check for new data every 5 seconds, forever, until stopped. Used for the live-style demo.
`--trigger availableNow`	Process everything waiting in Kafka in as many batches as needed, then **stop by itself**. Used by Airflow, tests and most demos (a finite task).
`--max-offsets-per-trigger N`	Cap on messages per batch (default 10,000). Small values (500) are used in recovery tests to create many batches.
`--starting-offsets earliest`	Only for the very first run; later runs always continue from the checkpoint.

8.7 Stopping safely (a real engineering story)

Spark has no official "stop between two micro-batches" button. The obvious approach, calling `query.stop()`, interrupts Spark's thread at *any* moment, even the instant a batch starts. In our end-to-end test this caused an **intermittent failure**: an `InterruptedException` inside a running batch. The data was still correct (the other 17 checks passed) because writes are idempotent, but a "graceful stop" that sometimes is not graceful is not good enough.

The fix is a small protocol (`order_stream.py`):

SRC/STREAMING/ORDER_STREAM.PY (SIMPLIFIED)

```
STATE = {"in_batch": False, "stopping": False}

def process_batch(batch_df, batch_id):
    STATE["in_batch"] = True              # 1) say "I am in a batch" FIRST ...
    if STATE["stopping"]:                 # 2) ... then look whether a stop was requested
        STATE["in_batch"] = False
        raise StopRequested("batch was not started and will run after the restart")
    ... do the real work ...

# main loop, when the stop file appears:
STATE["stopping"] = True                  # a batch that starts from now on refuses to start
wait until STATE["in_batch"] is False     # a running batch finishes normally
if the query is still active (idle): query.stop()
```

Because "in batch" is set before the stop flag is read, there is no gap: either the batch sees the stop request and does **nothing**, or the stopper sees the batch running and waits for it. A batch can therefore never be interrupted while it is writing. The refused batch makes Spark log an error block that contains the word `StopRequested`; this is the **expected signature**, followed by the line `STOPPED_AT_BATCH_BOUNDARY`. After the change, three consecutive recovery runs passed 18 of 18 checks, and the final end-to-end run passed.

> **What you will see in the demo**
>
> When you stop Spark, the log may show `ERROR` lines and a traceback ending with `StopRequested: stop requested: batch 3 was not started and will run after the restart`. Do not panic: that is the deliberate safe stop. A real problem would **not** contain `StopRequested`.

8.8 Commands for Spark

COMMANDS

```
.\scripts\run_stream.ps1 -Trigger availableNow     # finite run: process the backlog, then exit
.\scripts\run_stream.ps1                           # continuous run (stays open)
.\scripts\stop_stream.ps1                          # ask a running stream to stop safely
$py scripts\parquet_readback.py                    # read the Parquet files back (works on Windows)
```

REAL OUTPUT · RUN_STREAM.PS1 -TRIGGER AVAILABLENOW (RE-INGESTING 199 ORDERS)

```
... INFO order_stream BATCH_SUMMARY {"batch_id": 10, "input_rows": 199, "valid_rows": 199, "invalid_rows": 0,
    "in_batch_duplicates": 0, "parquet_rows_written": 199,
    "postgres": {"attempted": 199, "inserted": 0, "retry_duplicates": 199, "source_duplicates": 0, ...
... INFO order_stream Stream stopped.
```

Meaning: 199 messages arrived in batch 10 (it continued from the checkpoint: batch ids are never reused). **None were inserted**: all 199 were recognised as repeats already in the warehouse. The warehouse still had 99,427 rows. This is idempotency, proved on real data (chapter 16).

9. Stage 5: Data quality, duplicates and the dead-letter queue

9.1 Three promises

1. **Nothing is dropped silently.** A rejected record is stored with the reason and the original data.
2. **Empty is not wrong.** A missing delivery date is normal for an order not yet delivered. We reject only *contradictions*.
3. **Repeating is safe.** Sending the same data again never creates a duplicate row in the warehouse.

9.2 The rules (`config/quality_rules.yaml`, `src/processing/validators.py`)

Each rule was chosen because of something we *measured* in the real data (chapter 3), not guessed. Severity **reject** = goes to the dead-letter queue; **warn** = counted but the order continues.

Rule	Plain meaning	Real result (99,441 orders)
`status_in_allowed_set`	status must be one of the 8 values seen in the data	0 rejected
`purchase_date_in_range`	purchase date must be between 2016-01-01 and 2018-12-31	0
`amounts_not_negative`	price, freight, payment total must be ≥ 0	0
`approved_not_before_purchase`	approval cannot precede the purchase	0
`delivered_not_before_purchase`	delivery cannot precede the purchase	0 (used in Demo 3)
`delivered_status_has_date`	status "delivered" requires a delivery date	**8 rejected**
`delivered_date_matches_status`	a delivery date requires status "delivered"	**6 rejected** (all "canceled")
`payment_total_positive` (warn)	an order with payment rows should have a total > 0	3 warnings, not rejected

In addition the mapping step rejects empty required fields and wrong types (0 found in the real data). Total: **14 rejected = 8 + 6**, which is why 99,441 − 14 = 99,427.

> **Why these 14 are rejected**
> An order marked "delivered" with no delivery date would break the delivery-time KPI; a "canceled" order that nevertheless has a delivery date contradicts itself. They are **not deleted**: they sit in the dead-letter files with their full original data, so a human can inspect them.

9.3 What a dead-letter record looks like

REAL OUTPUT · DATA/DEAD_LETTER/INGEST/FULL-RUN-01.JSONL (ONE OF THE 14, SHORTENED)


```
{ "dlq_id": "bafe2d00-e461-4d9a-a417-959e5da272b6",
  "stage": "ingest",
  "reason_code": "INCONSISTENT",
  "reason_detail": "delivered_date_matches_status: status is 'canceled' but delivered_customer_ts is set",
  "record_uid": "92fc65a4424b55837a4389c5239ea75a",
  "ingest_run_id": "full-run-01",
  "raw_payload": { "order_id": "65d1e226...", "order_status": "canceled",
                   "order_delivered_customer_date": "2016-11-08 10:58:34", ... },
  "rejected_at": "2026-10-15T16:39:30.603849+00:00" }
```


Field	Meaning
`stage`	Where it was rejected: `ingest` (producer) or `stream` (Spark).
`reason_code`	A short category: `MISSING_REQUIRED`, `TYPE_ERROR`, `RANGE_ERROR`, `INCONSISTENT`, `MALFORMED_JSON`, `SCHEMA_MISMATCH`.
`reason_detail`	A sentence naming the rule and the values.
`raw_payload`	The original data, so nothing is lost.
`rejected_at`	UTC time stamp.

Files: `data/dead_letter/ingest/<run>.jsonl` and `data/dead_letter/stream/<stream>-batch-N.jsonl`. They can be loaded into the table `core.dead_letter_records` (`python -m src.warehouse.load_reference --only dlq`). Two situations are tested with **real Spark**: broken JSON gets `MALFORMED_JSON`; a JSON message missing `record_uid` or `order_id` gets `SCHEMA_MISMATCH` (`docs/evidence/04_spark/malformed_message_test.txt`, 7 of 7 checks).

9.4 The three kinds of "same record" (the most asked question)

Kind	How we detect it	What we do
Retry / replay duplicate	Same `record_uid` (same file, same row)	Do nothing; count it as `retry_duplicates`.
Source duplicate	Different `record_uid` but same `business_hash` (same `order_id` in two different rows)	Keep the first; log the other once in `core.duplicate_records`.
Re-ingestion	Same file ingested again under a new run id	Same `record_uid`s, so every record is a retry duplicate: nothing changes.

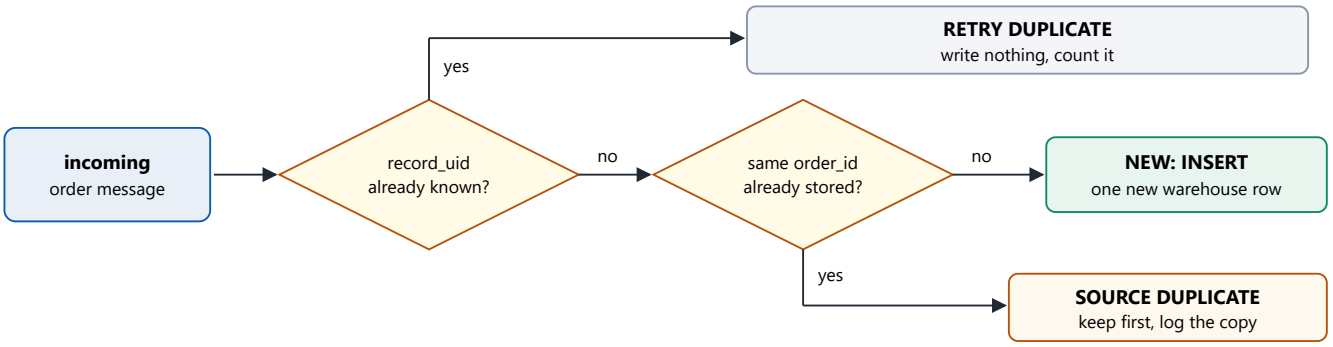


Figure 9.1: How the PostgreSQL writer classifies every incoming order.

> **Analogy: a library returns desk**
> Someone returns a book. If its barcode is already in the "returned" list, you ignore it (retry duplicate). If it is a different barcode but the same title that you already hold, you keep the first and note the second in a "spare copies" list (source duplicate). Otherwise you shelve it (new).

> **Honest note about the real data**
> The real orders file contains **no** source duplicates (every `order_id` is unique). Source duplicates are demonstrated with a small controlled test file (Demo 3). Retry duplicates, however, occur for real: whenever the same orders are replayed (we did it on purpose: 99 + 199 + 295 + ... repeats), and every one was ignored.

9.5 A worked example: the Demo 3 test file

A tiny CSV in the **real orders schema** (`data/test_inputs/demo3_records.csv`, using a real customer) with three rows:

Row	Order id	What is special	Expected and observed
1	`demo3_order_valid_001`	valid	published and stored
2	`demo3_order_invalid_002`	delivered on 2017-09-01, **before** the purchase on 2017-10-03	rejected: `RANGE_ERROR`, rule `delivered_not_before_purchase`
3	`demo3_order_valid_001`	exact copy of row 1	published (different `record_uid`); warehouse keeps row 1 and logs the copy as a source duplicate

Real result of ingesting it **twice** (15 of 15 automated checks passed, `docs/evidence/05_data_quality/quality_idempotency_test.txt`):

Layer	After 1st ingestion	After 2nd ingestion
Kafka messages	2	4
Parquet rows (distinct `record_uid`)	2 (2)	4 (2)
PostgreSQL `core.orders`	**1**	**1**
`core.duplicate_records`	1	1 (not logged twice)
2nd Spark batch result	`retry_duplicates: 2, inserted: 0, source_duplicates: 0`	

> **A bug this test found before it happened**
> While designing this test we noticed that a discarded copy is *not* in `core.orders`, so re-ingesting it would have been logged as a brand-new source duplicate every time. We fixed the writer: a `record_uid` already present in `core.duplicate_records` now counts as a retry, and a unique index protects the table.

9.6 Parquet and PostgreSQL behave differently on purpose

	Parquet (landing layer)	**PostgreSQL (warehouse)**
Purpose	Keep everything Spark consumed, for audit and replay	One clean row per order for analytics
A repeat arriving in a *later* batch	**Kept** (append-only)	Ignored (`retry_duplicates`)
A repeat inside the *same* batch	Removed (in-batch dedup)	(never arrives)
"How many orders?"	`count(DISTINCT record_uid)` = 99,427	`count(*)` = 99,427
Total rows today	100,120	99,427

So Parquet can contain **more rows** than PostgreSQL, and the difference is exactly the repeats. Reconciliation (chapter 15) reports and explains it.

10. Stage 6: PostgreSQL (the warehouse)

10.1 The structure

PostgreSQL holds two **schemas** (folders for tables): `core` (the real tables) and `analytics` (views, i.e. saved questions). Created by `sql/schema.sql` and `sql/analytics.sql` through `scripts/init_db.ps1`. This is **repeatable**: every statement is `CREATE ... IF NOT EXISTS` or `CREATE OR REPLACE`, so running it again never deletes data.

Table	What it holds	Rows today
`core.orders`	**The warehouse**: one row per order, with all typed fields, USD value, provenance and the original columns as JSON.	99,427
`core.duplicate_records`	Source-duplicate copies that were discarded.	0 (real data has none)
`core.dead_letter_records`	Rejected records loaded from the dead-letter files.	26 lines for 14 distinct records
`core.ingestion_runs`	One row per producer run (read, valid, rejected, published...).	5 production runs loaded
`core.api_cache`, `core.fx_rates`	Raw API responses; one exchange rate per date.	8 responses; 544 dates
`core.order_items`, `order_reviews`, `dim_products`, `dim_sellers`, `geo_zip`	Reference tables loaded in batch (not streamed).	112,650 / 99,224 / 32,951 / 3,095 / 19,010

10.2 The key table: `core.orders`

- **Primary key** `record_uid`: no two rows may share it.
- **Unique** `order_id`: the natural key, one row per order.
- Typed columns: timestamps are `TIMESTAMP`, money is `NUMERIC(12,2)` (exact decimals, never floating point), the zip prefix is `TEXT` to keep leading zeros (e.g. `01037`).
- Provenance columns: `source_file_sha256`, `source_row_number`, `ingest_run_id`, `kafka_partition`, `kafka_offset`, `processed_at`.
- API columns: `fx_rate_brl_usd`, `fx_match_type`, `payments_value_usd`, `api_endpoint`, `api_http_status` ...
- Indexes on purchase time, state and status (they make the dashboard queries fast).

10.3 The safe write (`src/warehouse/load.py`)

For every micro-batch, Spark hands the rows to `upsert_orders()`, which runs **inside one transaction** (all-or-nothing):

1. Copy the batch into a temporary *staging* table (it disappears at the end of the transaction).
2. Find the **known** IDs: `record_uid`s already in `core.orders` or already logged in `core.duplicate_records` ⇒ these are **retry duplicates** (counted only).
3. Among the rest, rank rows by `business_hash` and keep the first (lowest source row number).

4. Log the others (or a copy of an order that is already stored) in `core.duplicate_records` as **source duplicates**.
5. `INSERT` the remaining first rows into `core.orders`.

SRC/WAREHOUSE/LOAD.PY (THE INVARIANT THAT KEEPS US HONEST)

```
accounted = result["inserted"] + result["retry_duplicates"] + result["source_duplicates"]
if accounted != result["attempted"]:      # should never happen; make it loud if it does
    log.error("PostgreSQL batch %s: %d rows attempted but only %d accounted for", ...)
```

Every attempted row must be accounted for as exactly one of: inserted, retry, or source duplicate. (Seven integration tests on a real PostgreSQL check each case.) Database values are always passed as **parameters**, never pasted into SQL text (this prevents SQL injection).

> **Why a transaction?**
> If something fails in the middle of a batch, the whole batch is rolled back: the database never contains "half a batch". Together with idempotency, re-running that batch later gives the correct result.

10.4 Reference loaders and run summaries

COMMAND

```
$py -m src.warehouse.load_reference                 # everything
$py -m src.warehouse.load_reference --only runs dlq api    # selected parts
```

REAL OUTPUT · DOCS/EVIDENCE/07_POSTGRESQL/LOAD_REFERENCE_RUN2.TXT

```
products -> {'products_in_file': 32951, 'untranslated_category_rows': 623}
sellers  -> {'sellers_in_file': 3095}
items    -> {'order_items_in_file': 112650}
reviews  -> {'reviews_in_file': 99224, 'repeated_review_id_order_id_pairs_skipped': 0}
geo      -> {'geolocation_rows_read': 1000163, 'rows_outside_brazil_ignored': 42, 'zip_prefixes_loaded':
19010}
runs     -> {'ingestion_run_files_seen': 13, 'runs_loaded': 3}
dlq      -> {'dead_letter_records_loaded': 16, 'run_groups_replaced': 3}
api      -> {'api_cache_files': 7, 'fx_rate_rows_seen': 786}
```

Running it a second time changed nothing (idempotent): products, sellers etc. use `ON CONFLICT DO NOTHING`; dead letters are replaced per run, never duplicated. The loader reads only runs of the production topic, so test and demo runs never pollute the real warehouse. The geolocation file (1,000,163 rows) is read in chunks and reduced to one average point per zip prefix (19,010 points).

10.5 The analytics views (`sql/analytics.sql`)

A **view** is a saved question. The dashboard just reads these views. Definitions used everywhere:

- **Revenue** = `payments_value_total` (what customers paid, including freight and after vouchers).
- **Delivery time** = delivered date − purchase date (only for delivered orders).
- **Late** = delivered after the promised (estimated) date.
- **USD** = estimate (BRL × ECB rate of the purchase day).

View	Answers	Real values

`revenue_summary`	Totals	99,427 orders; revenue 16,006,744.62 BRL; items 13,589,748.34; freight 2,251,677.40; average order 160.99 BRL; USD estimate 4,768,687.37
`order_status_distribution`	How many per status	delivered 96,470 (97.03 %); shipped 1,107; canceled 619; unavailable 609; invoiced 314; processing 301; created 5; approved 2
`orders_per_day`, `orders_per_month`	Growth over time	2016-09: 4 orders → peak 2017-11: 7,543 orders (Black Friday month) → 2018-08: 6,512
`payment_type_mix`	Orders using each payment type (an order can use several)	credit_card 76,492; boleto 19,784; voucher 3,866; debit_card 1,527; not_defined 3
`late_delivery_summary`	Late rate	96,470 delivered; 7,826 late = **8.11 %**; average delivery 12.56 days
`delivery_time_by_month`	Average and median days	e.g. 2018-08: avg 7.73, median 7.01 days
`orders_by_state`	Where customers are	SP 41,738; RJ 12,849; MG 11,634; RS 5,465; PR 5,045
`top_product_categories`	Best categories (English names)	bed_bath_table 11,115; health_beauty 9,668; sports_leisure 8,639
`review_score_distribution`	Review scores 1-5	1 star: 11,419; 2: 3,151; 3: 8,178; 4: 19,142; **5 stars: 57,320**
`api_enrichment_summary`	API health on the data	99,427 enriched = 100 %; 24,112 on previous business day

Sanity check that the numbers agree with the profile: delivered 96,470 = 96,478 − 8 rejected; canceled 619 = 625 − 6 rejected. The data was neither lost nor invented.

10.6 The quality checks on the stored data

COMMAND

```
docker cp sql\quality_checks.sql streampulse-postgres:/tmp/qc.sql
docker exec streampulse-postgres psql -U streampulse -d streampulse -f /tmp/qc.sql
```

REAL OUTPUT · DOCS/EVIDENCE/07_POSTGRESQL/QUALITY_CHECKS_OUTPUT.TXT

```
 rows in core.orders     | 99427      distinct record_uid | 99427
 distinct order_id       | 99427      distinct business_hash | 99427
 status outside the allowed set         | 0
 duplicate order_id (problem_rows)      | 0
 delivery date but status not delivered | 0
 delivered before purchase              | 0
 negative amounts                       | 0
 delivered without delivery date        | 0
 orders without items (kept, count 0)   | 775        orders without payments (kept) | 1
```

Meaning: the four counts are identical (one row per order); every problem count is zero; the 775 item-less orders and the 1 payment-less order are kept on purpose and reported.

11. Stage 7: Airflow (running the steps as a workflow)

11.1 What Airflow is, and why it is here

So far you started each step by hand. In real companies, steps must run in the right order, automatically, with **retries when something temporarily fails** and **logs** to see what happened. A tool that does this is a **workflow orchestrator**. We use **Apache Airflow**.

In Airflow a workflow is a **DAG** (Directed Acyclic Graph): tasks joined by arrows meaning "this must finish first". "Acyclic" means no loops.

> **Analogy: a recipe card**
> A DAG is a recipe: "1 chop onions, 2 fry onions, 3 add tomatoes". Airflow is the cook's assistant who follows the card, redoes a step if the stove fails once, and writes down what happened.

> **Why the stream is NOT an Airflow task**
> An Airflow task must **end**. A continuous Spark stream (a batch every 5 seconds, forever) never ends. So Airflow runs Spark with `--trigger availableNow` ("process what is waiting, then finish"), and the replay is limited by a parameter. Say this in the viva: "*Airflow orchestrates finite tasks; the endless stream is not a DAG task.*"

11.2 How it is deployed

- A custom Docker image (`docker/airflow/Dockerfile`) based on Airflow 2.10.5 with Java and its own Python virtual environment for our code (so our packages never clash with Airflow's).
- **LocalExecutor** (runs tasks as processes on the same machine). Airflow's own bookkeeping lives in a **separate database named** `airflow` inside the same PostgreSQL (the `streampulse` data is untouched).
- Start: `.\scripts\airflow_up.ps1` ⇒ UI at `http://localhost:8080`, user `admin`, password = `AIRFLOW_ADMIN_PASSWORD` in your `.env`.
- DAG files: `airflow/dags/streampulse_batch_dag.py` and `streampulse_failure_demo_dag.py`.

11.3 The DAG `streampulse_batch`: nine finite tasks

Each task is a command run inside the Airflow container. A task starts only when the arrow's source succeeded.

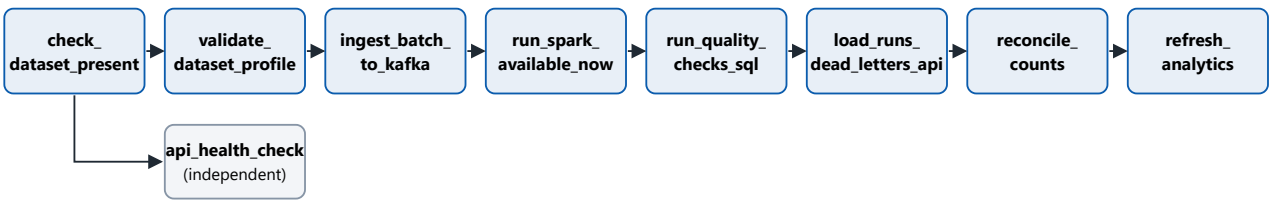


Figure 11.1: The batch DAG. Real graph: `docs/evidence/08_airflow/airflow_streampulse_batch_graph.png`.

Task	What it runs	Seconds in the real run
`check_dataset_present`	all 9 CSV files exist	1
`validate_dataset_profile`	files still match the recorded SHA-256 fingerprints	2
`api_health_check`	one real request to Frankfurter (independent branch)	1
`ingest_batch_to_kafka`	producer with `--limit {{ params.limit }}` (default 500; we used 300)	5
`run_spark_available_now`	Spark with `availableNow` into Parquet and PostgreSQL	70
`run_quality_checks_sql`	`scripts/run_quality_checks.py` : fails if any problem count > 0	8
`load_runs_dead_letters_api`	loads run summaries, dead letters, API cache	1
`reconcile_counts`	`scripts/reconcile.py` : fails on an unexplained difference	12
`refresh_analytics`	re-applies the views (never drops data)	0.4

Settings on every task: **2 retries**, **30 s** delay between retries, **time limit** (20 minutes; 30 for Spark). The DAG allows **one run at a time** (`max_active_runs=1`) because two runs would share one Spark checkpoint. Every step is safe to repeat, because replaying the same orders produces the same `record_uid` s.

11.4 The real run

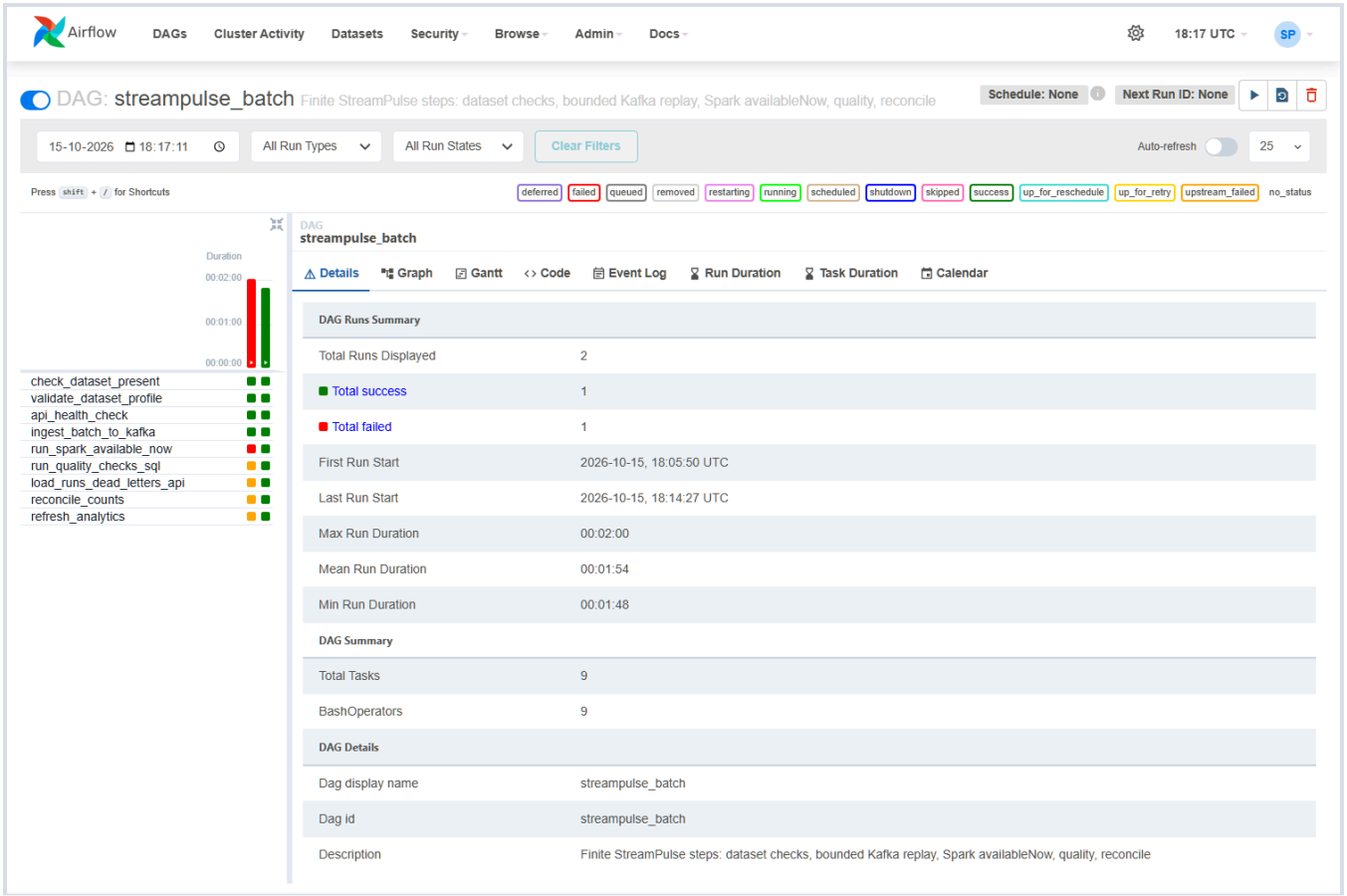


Figure 11.2: The real Airflow UI (grid view). The left bar is the **first run (red: it failed)**, the right bar is the **second run (all green: success)**. Rows are the nine tasks.

REAL OUTPUT · DOCS/EVIDENCE/08_AIRFLOW/DAG_RUN2_STREAMPULSE_BATCH.TXT

```
Triggered streampulse_batch run_id=manual__20261015T181422 conf={"limit": 300}
23:44:40  run state: running
   ...
23:46:22  run state: success
run_spark_available_now    success   18:14:38 -> 18:15:48
run_quality_checks_sql     success
reconcile_counts           success
FINAL STATE: success
```

Inside the logs: the producer published 295 valid orders; Spark batch 11 received 590 messages (295 from the earlier failed attempt's ingestion + 295 new), inserted **0** and counted 295 retry duplicates; the quality checks printed `RESULT: all quality checks passed`; reconciliation printed `RESULT: all differences are zero or explained`.

The failed first run (kept as evidence, and why it matters)

Real failure: "Permission denied"

`run_spark_available_now` failed 3 times (retries working) with `java.io.FileNotFoundException: /opt/streampulse/data/checkpoints/orders/offsets/.11....tmp (Permission denied)`. **Cause:** the production checkpoint folder was created earlier by the Spark container as the **root** user, but Airflow tasks ran as a different user who was not allowed to write there. **Fix:** the Airflow container now runs as root (documented trade-off for a demo; a real system would share one user or use volumes). The other tasks in that run succeeded, and the downstream tasks were marked `upstream_failed` (they correctly refused to run after a failure).

11.5 The retry and failure demonstration (`streampulse_failure_demo`)

To *show* retries, a second DAG runs the real producer against a Kafka address that does not exist (`kafka-does-not-exist:9092`). This simulates a real outage of a real dependency; no mock service is used.

REAL OUTPUT · DOCS/EVIDENCE/08_AIRFLOW/DAG_FAILURE_DEMO_RUN.TXT

```
attempt=1.log: Starting attempt 1 of 3 | ... KafkaTimeoutError: Unable to bootstrap from kafka-does-not-exist:9092
attempt=2.log: Starting attempt 2 of 3 | ... KafkaTimeoutError: Unable to bootstrap from kafka-does-not-exist:9092
attempt=3.log: Starting attempt 3 of 3 | ... KafkaTimeoutError: Unable to bootstrap from kafka-does-not-exist:9092
ingest_from_unreachable_kafka  failed
downstream_never_runs          upstream_failed
FINAL STATE: failed
```

Meaning: three attempts (1 original + 2 retries), then the task is `failed`, its follower is `upstream_failed`, and the DAG run is `failed`. **This failure is the intended result**; the DAG is kept paused so it never runs by accident.

11.6 Commands

COMMANDS

```
.\scripts\airflow_up.ps1                                                    # start Airflow (first time builds
the image)
.\scripts\airflow_run_dag.ps1 -Dag streampulse_batch -Conf '{"limit": 300}'   # trigger, wait, print every
task state
.\scripts\airflow.ps1 dags list                                             # any Airflow CLI command
.\scripts\run_tests_airflow.ps1                                             # the 8 DAG structure tests (inside
the container)
.\scripts\airflow_down.ps1                                                  # stop (nothing is deleted)
```

`airflow_run_dag.ps1` exits with 0 only if the run ends in `success`.

12. Stage 8: The Streamlit dashboard (every page explained)

12.1 How the dashboard works

Start it with `.\scripts\run_dashboard.ps1` and open `http://localhost:8501`. It is three separate pieces:

File	Responsibility
`src/dashboard/queries.py`	All database and Parquet access. Every number on the dashboard comes from here. Errors are logged fully for the developer but only a short safe message reaches the user (no passwords, no host names).
`src/dashboard/status.py`	The status lights: **real checks**, not decoration.
`src/dashboard/app.py`	Only the screen layout: tables, charts, text.

- Data is cached for 30 seconds; the sidebar button **Refresh data** clears the cache.
- Customer identifiers are never displayed.
- Money columns from PostgreSQL arrive as exact decimals; they are converted to normal numbers before charting.
- For demos: `.\scripts\run_dashboard.ps1 -Demo 3` shows the isolated demo database instead.

> **Rule: no fake green lights**
> A status light is green **only if a real check just succeeded**. If a service is down it is red; if the check cannot be made it is grey ("unknown"). This was verified by really stopping Kafka and PostgreSQL (section 12.9).

12.2 Page 1: Overview and status

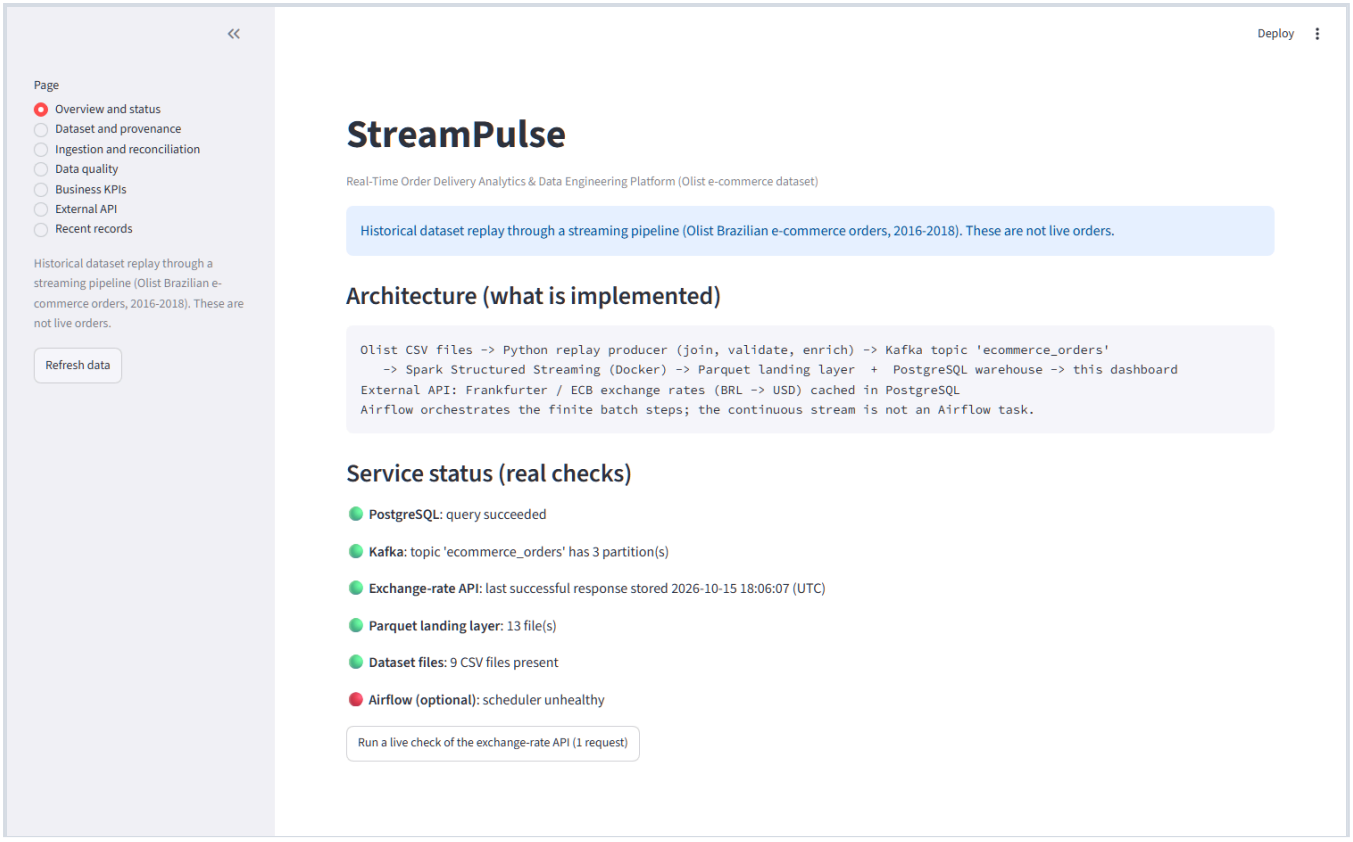


Figure 12.1: Overview and status (real screenshot).

Element	What it tells you
Blue banner	The honest framing: historical replay, not live orders. It is repeated in every page's sidebar.
Architecture box	The implemented data flow in four lines.
PostgreSQL light	Runs `SELECT 1`. Green = the database answered.
Kafka light	Asks the broker for the topic's metadata. Green text: "topic 'ecommerce_orders' has 3 partition(s)".
Exchange-rate API light	Reads the last successful API response *stored in PostgreSQL* and shows its time (no new call).
Parquet landing layer light	Lists the files (e.g. "12 file(s)").
Dataset files light	Counts the CSV files (must be 9).
Airflow (optional) light	Calls Airflow's health URL. Red "not running" is normal when you did not start it; it is optional.
Button "Run a live check of the exchange-rate API"	Makes exactly **one** real request and shows HTTP status and number of rates.

12.3 Page 2: Dataset and provenance

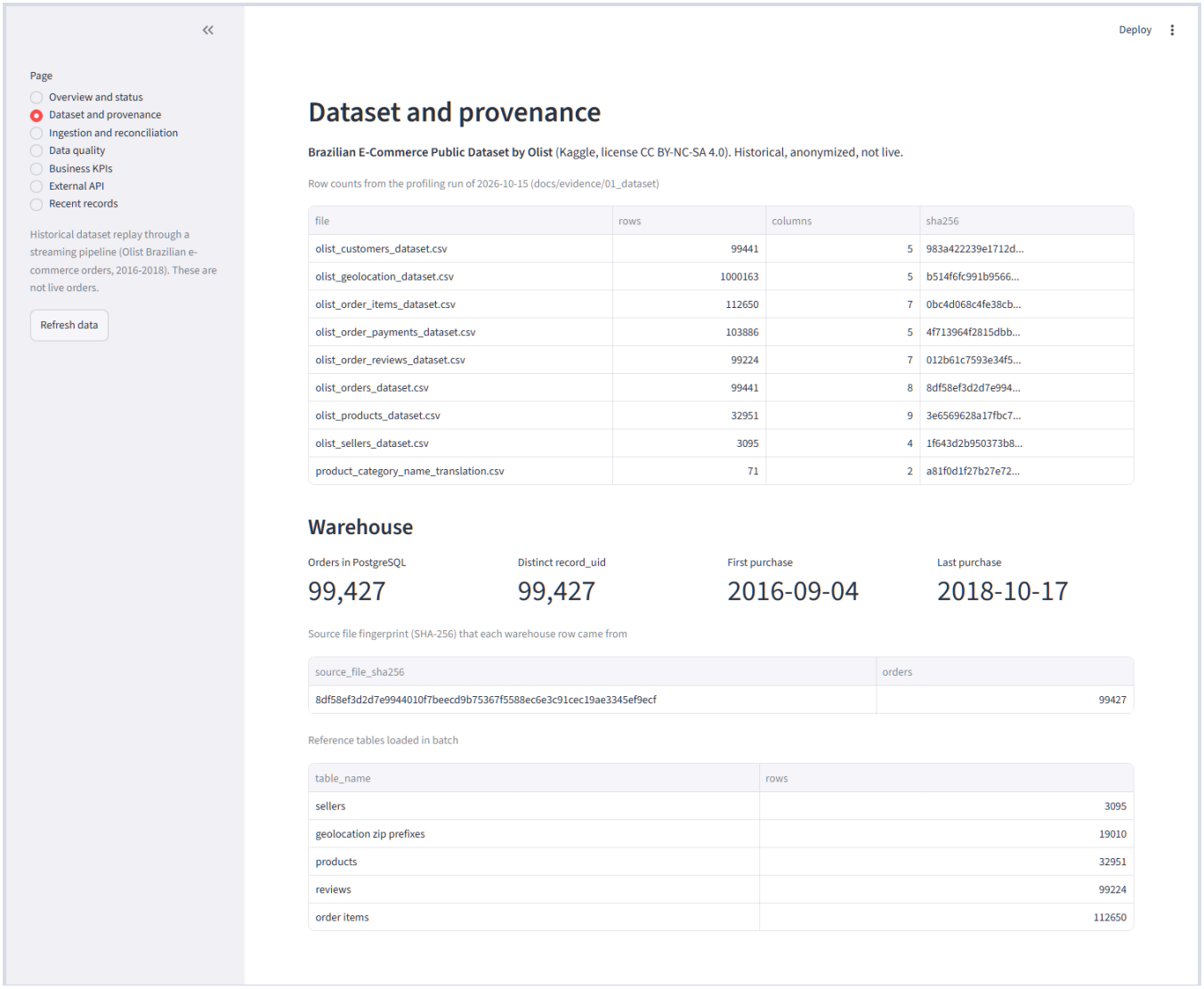


Figure 12.2: Dataset and provenance.

- **First table:** the 9 source files with rows, columns and the start of their SHA-256 fingerprints (from the profiling run of 2026-10-15).
- **Warehouse cards:** 99,427 orders in PostgreSQL = 99,427 distinct `record_uid` (one row per order); first purchase 2016-09-04, last 2018-10-17.
- **Fingerprint table:** all 99,427 warehouse rows came from **one** source file with the fingerprint `8df58ef3...`, the same one recorded on day one. This is the provenance proof: the data came from the untouched Olist orders file.
- **Reference tables:** sellers 3,095; geolocation zip prefixes 19,010; products 32,951; reviews 99,224; order items 112,650.

12.4 Page 3: Ingestion and reconciliation

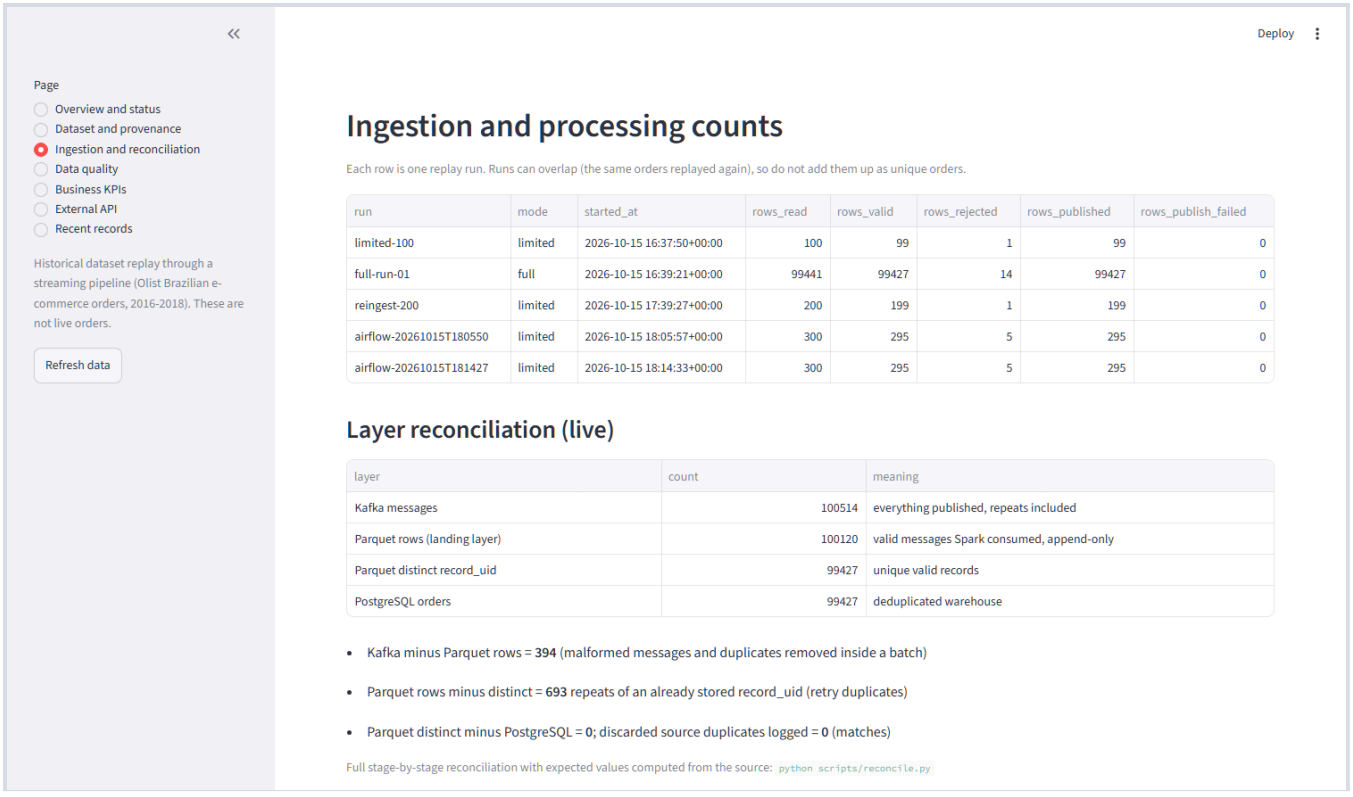


Figure 12.3: Ingestion and reconciliation (real screenshot).

Top table: one row per producer run. Read `full-run-01`: it read 99,441 rows, 99,427 were valid, 14 rejected, 99,427 published, 0 failed to publish. The runs **overlap** (the same first orders were replayed several times on purpose), so never add the rows up to get "unique orders".

Layer reconciliation:

Layer	Count	How to read it
Kafka messages	100,514	Everything published, repeats included (99,427 unique + 1,087 repeats).
Parquet rows	100,120	What Spark consumed and kept (append-only).
Parquet distinct `record_uid`	99,427	The unique valid orders.
PostgreSQL orders	99,427	The deduplicated warehouse.

- **Kafka minus Parquet rows = 394:** repeats that Spark removed *inside one batch* (99 in the first batch + 295 in batch 11).
- **Parquet rows minus distinct = 693:** repeats that arrived in a *later* batch (199 + 295 + 199); Parquet keeps them, PostgreSQL ignores them.
- **Parquet distinct minus PostgreSQL = 0** and "discarded source duplicates = 0": they match (the real data has no source duplicates).

The line at the bottom reminds you that the full source-derived reconciliation is `scripts/reconcile.py` (chapter 15).

12.5 Page 4: Data quality

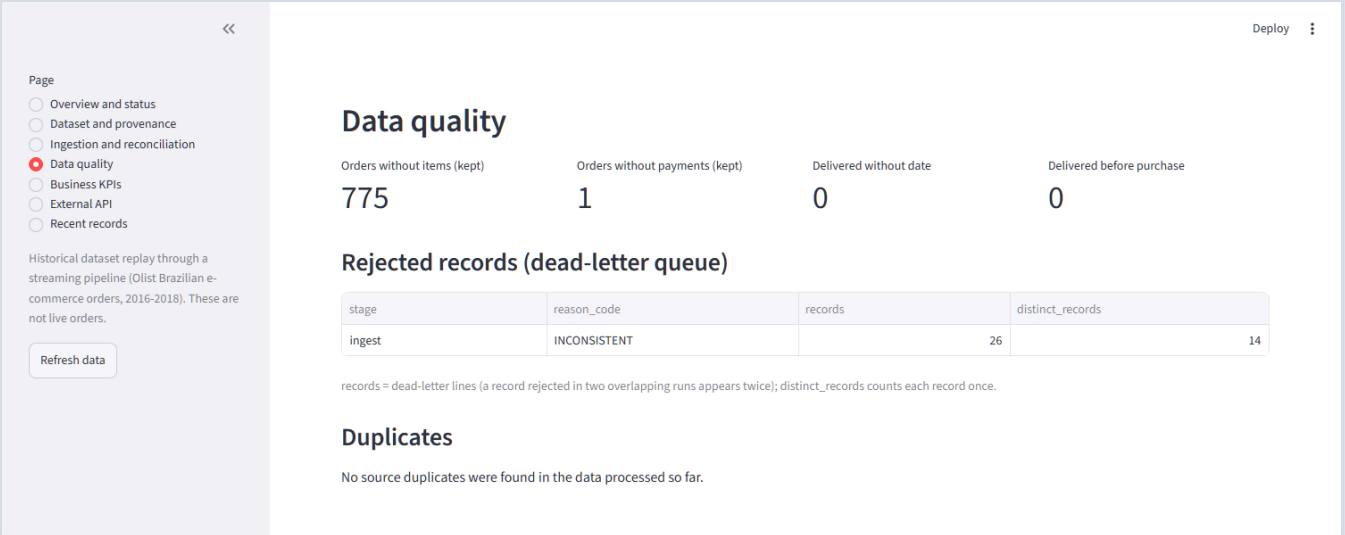


Figure 12.4: Data quality (real screenshot).

- **Orders without items (kept): 775** and **without payments (kept): 1.** They are incomplete but real; we keep them and show the count.
- **Delivered without date: 0** and **Delivered before purchase: 0.** These two problem counters must be zero in the warehouse (the bad ones were rejected before arriving).
- **Rejected records (dead-letter queue):** stage `ingest`, reason `INCONSISTENT`, **26** records, **14** distinct. 26 lines because overlapping runs rejected the same 14 records again (one line per run); 14 is the number of different orders.
- **Duplicates:** "No source duplicates were found" (true for the real data).

12.6 Page 5: Business KPIs

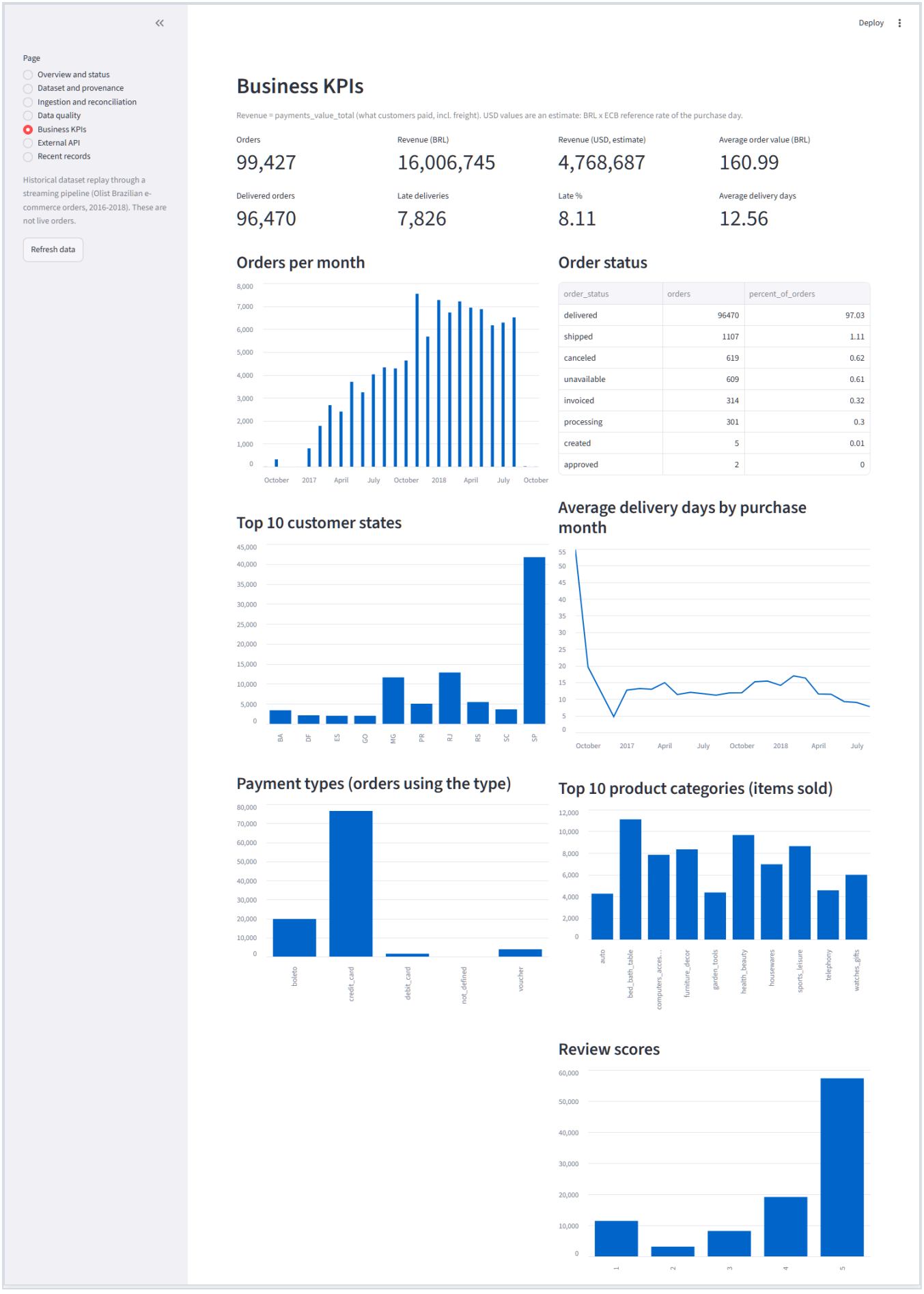


Figure 12.5: Business KPIs (real screenshot; the last chart continues below the image edge).

Metric / chart	What it means	Real value / reading
Orders	unique orders in the warehouse	99,427
Revenue (BRL)	sum of `payments_value_total` (what customers paid)	16,006,745
Revenue (USD, estimate)	BRL × ECB rate of each purchase day	4,768,687
Average order value	average payment of orders with a payment	160.99 BRL
Delivered orders / Late deliveries / Late %	delivered orders; delivered after the promised date; their share	96,470 / 7,826 / **8.11 %**
Average delivery days	delivery date − purchase date	12.56 days
Orders per month (bar)	business growth	starts at 4 orders (Sept 2016), grows, peaks in **Nov 2017 (7,543)**, about 6,000-7,000 per month in 2018
Order status (table)	share of each status	delivered 97.03 %; shipped 1.11 %; canceled 0.62 %
Top 10 customer states (bar)	where orders come from	SP (São Paulo) is far ahead: 41,738; then RJ 12,849; MG 11,634
Average delivery days by month (line)	how delivery speed changed	recent months are faster, e.g. Aug 2018 avg 7.73 days vs May 2018 11.42
Payment types (bar)	orders using each type (an order can use several)	credit_card 76,492 > boleto 19,784 > voucher 3,866 > debit_card 1,527
Top 10 categories (bar)	items sold per product category	bed_bath_table 11,115; health_beauty 9,668; sports_leisure 8,639
Review scores (bar)	how customers rated	5 stars: 57,320; 4: 19,142; 3: 8,178; 2: 3,151; 1: 11,419

> **How to word the USD number**
> Say "an **estimated** 4.77 million USD at the European Central Bank's rate on each purchase day". Customers paid in reais.

12.7 Page 6: External API

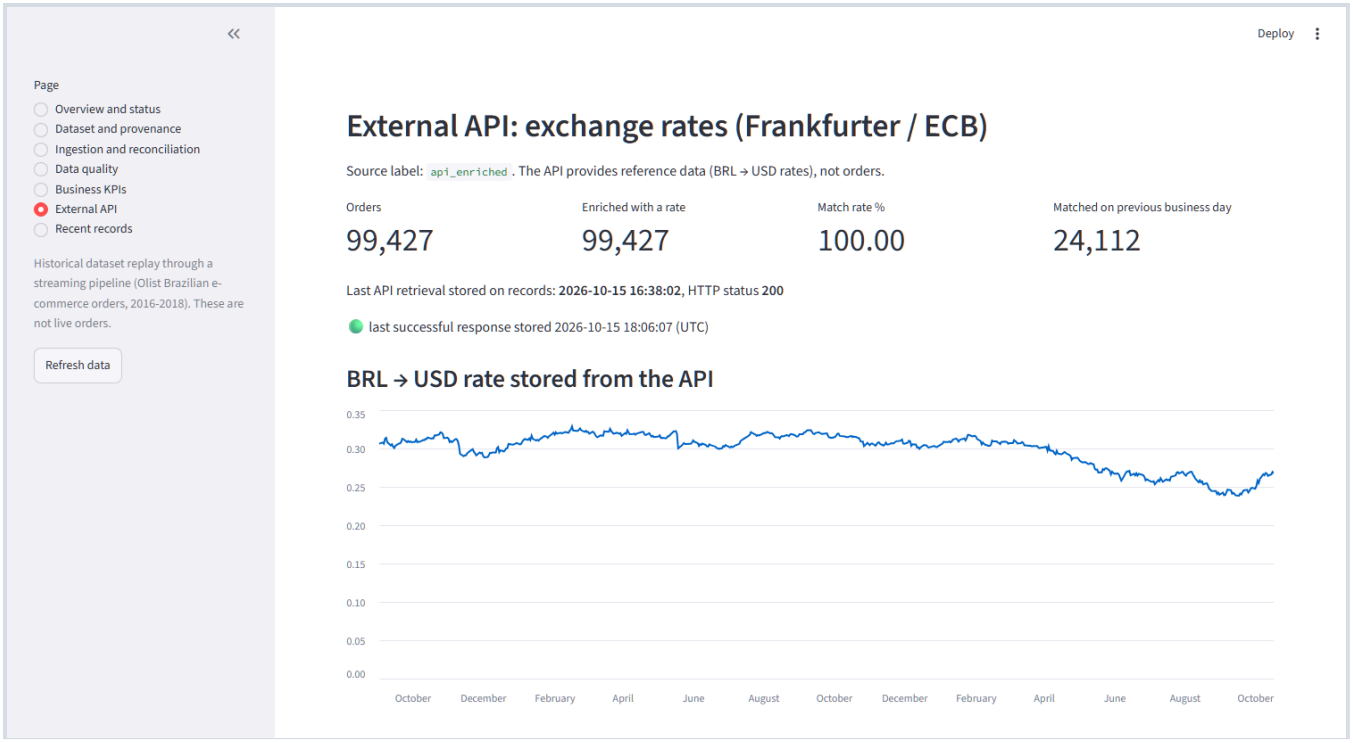


Figure 12.6: External API (real screenshot).

- **Orders 99,427 / Enriched with a rate 99,427 / Match rate 100.00 %:** every order received a USD value.
- **Matched on previous business day: 24,112:** orders bought on weekends/holidays, which have no ECB rate.
- **"Last API retrieval stored on records … HTTP status 200":** provenance of the data in the warehouse.
- **Green light:** the last successful response stored in PostgreSQL.
- **Line chart:** the real BRL→USD rate from Sept 2016 to Oct 2018 (about 0.24 to 0.33 dollars per real; it falls in 2018 as the real weakened).

12.8 Page 7: Recent records

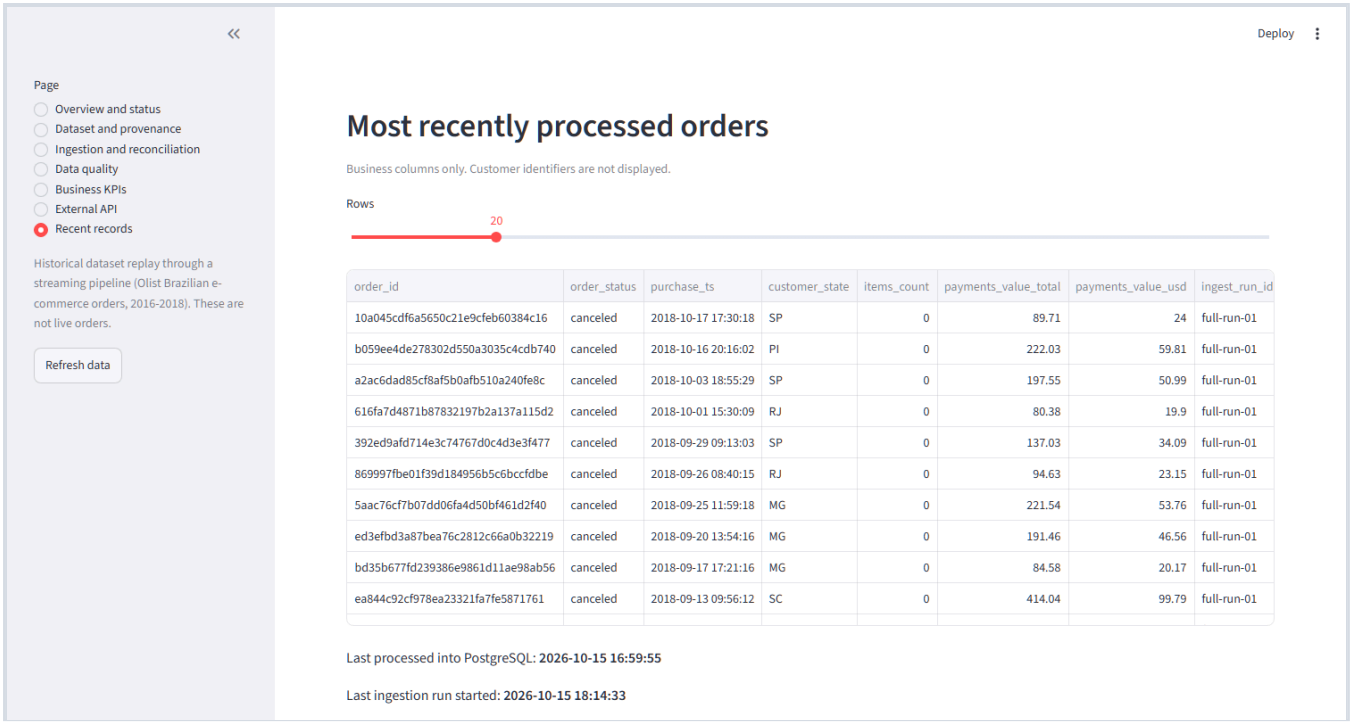
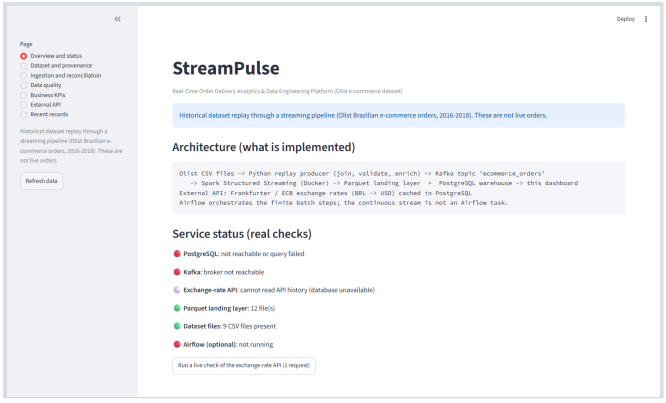


Figure 12.7: Most recently processed orders (real screenshot).

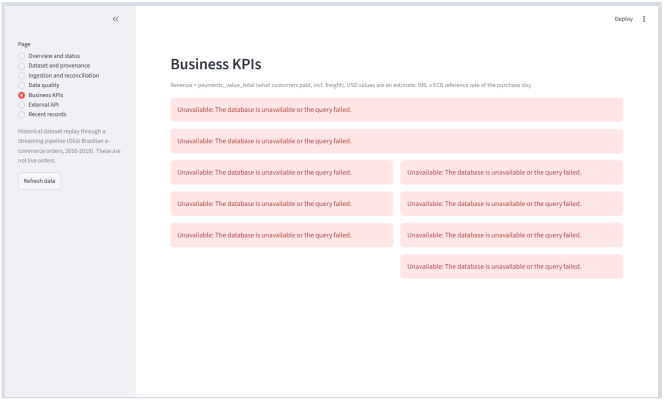
A table of the latest rows with business columns only: `order_id`, status, purchase time, state, number of items, payment in BRL and USD, which ingestion run brought it. A slider chooses how many rows (5-100). Below: **last processed into PostgreSQL** and **last ingestion run started**, i.e. how fresh the data is. (Customer identifiers are deliberately not shown.)

12.9 When services are really down

We stopped the Kafka and PostgreSQL containers on purpose and took real screenshots, then restored them (99,427 rows were intact afterwards).



Overview with Kafka and PostgreSQL stopped: both **red**; the API light is **grey** (cannot be known without the database); Parquet and dataset files are still **green** because those checks really passed.



KPI page with PostgreSQL stopped: **"Unavailable: The database is unavailable or the query failed."** No technical details leak.

Two dashboard bugs found by LOOKING at screenshots (not by tests)

1. The Kafka light was red while Kafka was running: a client option that this library version rejects raised an error, which my code reported as "down". (My first "down" tests passed for the wrong reason, so a test against the real broker was added.) **2.** Three pages printed Streamlit's own documentation: a line of the form `st.dataframe(x) if cond else st.write(y)` is a bare expression, and Streamlit displays the value of bare expressions. Fixed with a plain if/else, plus an automatic source check. Lesson: **always look at the real output.**

13. The folder and file map

Open the project folder and you will see this. Learn the **purpose** of each folder; in the viva you will be asked "where is X?".

Path	What it contains / why it exists
`Readme.md`	Front page of the project: what it is, how to start.
`.env` (you create it)	Passwords and settings. **Never committed** (listed in `.gitignore`). Template: `.env.example`.
`docker-compose.yml`	Defines every service (Kafka, PostgreSQL, Spark, Airflow): image, ports, volumes, health checks.
`requirements.txt`, `requirements-dev.txt`	Python packages for running / for testing. `pytest.ini` defines the six test markers.
`data/raw/`	The 9 Olist CSV files (git-ignored: licence and size).
`data/processed/parquet/orders/`	The Parquet landing layer (`batch_id=NNNNNNNN` folders).
`data/processed/spark_batches/orders.jsonl`	One JSON line per micro-batch: the `BATCH_SUMMARY` counts.
`data/checkpoints/orders/`	Spark's progress memory. **Never delete.**
`data/dead_letter/`	Rejected records as JSON lines: `ingest/` (producer) and `stream/` (Spark).
`config/field_mapping.yaml`	The map from Olist columns to message fields (single source of truth for the mapping).
`config/quality_rules.yaml`	The 8 data-quality rules.
`src/common/`	`config.py` (reads `.env`), `mapping.py`, `orders_source.py` (the join).
`src/api/`	`client.py` (HTTP + retry + cache), `enrich.py` (BRL→USD logic).
`src/producer/dataset_producer.py`	The Kafka producer (CLI).
`src/processing/`	`validators.py`, `classify.py` (pure rule functions), `deadletter.py`.
`src/streaming/`	`order_stream.py` (the Spark job, stop protocol), `transforms.py` (parse/flatten/validate in Spark).
`src/warehouse/`	`db.py` (connection), `load.py` (idempotent upsert), `load_reference.py` (items/products/sellers/reviews/geo).
`src/dashboard/`	`queries.py`, `status.py`, `app.py` (Streamlit).
`sql/`	`schema.sql` (tables), `analytics.sql` (11 views), `quality_checks.sql`, `analytics_samples.sql`.
`scripts/`	PowerShell and Python helpers: every command in chapter 14 (see below).
`airflow/dags/`	The two DAG files.
`docker/spark`, `docker/airflow`	Dockerfiles for the two custom images.

`tests/unit | integration | e2e`	Automated tests, by how much of the system they need.
`docs/`	Specs (00-07), architecture, setup guide, data dictionary, quality rules, test cases, viva questions, limitations, final report, task tracker, evidence register.
`docs/evidence/00..11`	Saved real outputs and screenshots, one folder per stage. Each file records its command, time and git commit.
`docs/phase_reports/`	What was done in each phase, including failures.

Script families

- **kafka_*.ps1** start/stop Kafka, list/describe topic, consume N messages, count per partition.
- **run_stream.ps1 / stop_stream.ps1** start and safely stop Spark. **init_db.ps1** creates the schema.
- **airflow_*.ps1** start, stop, run a DAG, any CLI command.
- **run_dashboard.ps1** starts Streamlit. **reconcile.py**, **run_quality_checks.py** verify the numbers.
- **profile_dataset.py**, **check_dataset_fingerprint.py** examine the data. **secret_scan.py**, **check_evidence_paths.py** guard the repository.
- **demo_setup / demo_load / demo3_cleanup / reset_demo_state** support the four demos on an isolated demo database.
- **spark_experiments.py, quality_experiment.py, measure_key_options.py, explain_rejected_count.py**: experiments that produced evidence.

14. The commands cookbook

Run every command from the **project root** in **PowerShell**. First, in each terminal: `$py = ".\.venv\Scripts\python.exe"`. Each entry gives: the command, *why*, what success looks like (taken from real runs) and what it means.

14.1 First-time setup

Command	Why	Expected output and meaning
`python -m venv .venv` `$py -m pip install -r requirements.txt`	Private Python environment with the exact packages.	"Successfully installed ...". If it fails, the Python version is wrong (we used 3.11).
`Copy-Item .env.example .env`	Create your settings file. Edit `POSTGRES_PASSWORD`.	A new `.env` file. Never share or commit it.
`.\scripts\check_env.ps1`	Are Python, Docker and the Kafka port ready?	Versions printed; port 9092 `reachable = True` only after Kafka is started.
`$py scripts\check_dataset_fingerprint.py`	Are the 9 CSVs the untouched originals?	`All 9 files match the fingerprints...` A mismatch means a file was altered or is wrong.

14.2 Infrastructure

Command	Why	Expected output and meaning
`.\scripts\kafka_start.ps1`	Start Kafka and create the topic.	`Kafka health: healthy` and the topic `ecommerce_orders` with 3 partitions. `docker ps` shows `(healthy)`.
`.\scripts\init_db.ps1`	Start PostgreSQL, create schemas, tables, views.	`schema OK`. Safe to repeat (nothing is dropped).
`.\scripts\kafka_describe_topic.ps1`	Show topic settings.	`PartitionCount: 3 ReplicationFactor: 1`.
`docker compose --profile spark build spark`	Build the Spark image once.	5-10 minutes the first time; ends with the image name. (Needed because Windows cannot run Spark directly.)

14.3 Ingestion

Command	Why	Expected output and meaning
`$py -m src.producer.dataset_producer --limit 100 --dry-run`	Rehearse: read, join, validate, enrich; publish nothing.	A JSON summary with `rows_published: 0`. Nothing reached Kafka.
`$py -m src.producer.dataset_producer --limit 200 --run-id demo1`	Publish the first 200 orders.	`rows_read 200, rows_valid 199, rows_rejected 1, rows_publish_failed 0`. One order is bad and goes to the dead-letter file.

`$py -m src.producer.dataset_producer --full`	Replay all orders (about 100 s).	`rows_read 99441, rows_valid 99427, rows_rejected 14`. Without `--limit` or `--full` the producer refuses to run (so nobody replays 100k orders by accident).
`.\scripts\kafka_consume_n.ps1 -Count 1 -Partition 0`	Look at a real message.	One JSON envelope (chapter 5).
`.\scripts\kafka_partition_counts.ps1`	Messages per partition.	Three numbers that are nearly equal (33.4 / 33.4 / 33.2 %). Total = messages published.
`$py scripts\api_demo_call.py --sample --no-cache`	One real API request.	`HTTP : 200`, `Rows : 5`.
`$py scripts\api_demo_call.py --simulate-failure`	Show the retry logic.	`API failed after 3 attempts` (waits of 1 s and 2 s).

14.4 Spark, Parquet, PostgreSQL

Command	Why	Expected output and meaning
`.\scripts\run_stream.ps1 -Trigger availableNow`	Process everything waiting in Kafka, then stop.	One `BATCH_SUMMARY {...}` line per batch, then `Stream stopped.` Counts: input_rows, valid, invalid, in-batch duplicates, inserted, retry duplicates, source duplicates.
`.\scripts\run_stream.ps1`	Continuous mode: a batch every 5 s.	A summary whenever new messages arrive; waits otherwise.
`.\scripts\stop_stream.ps1`	Safe stop.	The running batch finishes first; the log shows `STOPPED_AT_BATCH_BOUNDARY`. Never kill the container.
`$py scripts\parquet_readback.py`	Read the Parquet files back.	Row count, distinct `record_uid`, schema, 3 sample rows. Rows ≥ distinct (Parquet keeps repeats).
`docker exec streampulse-postgres psql -U streampulse -d streampulse -c "SELECT count(*), count(DISTINCT record_uid) FROM core.orders"`	Count warehouse orders.	`99427 | 99427` after a full run: one row per order.
`$py scripts\run_quality_checks.py`	Run the SQL quality checks.	A table of problem counts, then `RESULT: all quality checks passed`. Exit code 1 if any is above 0.
`$py scripts\reconcile.py`	Compare every layer with the source.	`RESULT: all differences are zero or explained`; exit code 0 (see chapter 15).

14.5 Airflow, dashboard, tests

Command	Why	Expected output and meaning
`.\scripts\airflow_up.ps1`	Start Airflow.	`Airflow is healthy`; UI at port 8080.
`.\scripts\airflow_run_dag.ps1 -Dag streampulse_batch -Conf '{"limit": 300}'`	Run the workflow once.	Task states, then `FINAL STATE: success` in about 2 minutes.

`.\scripts\run_dashboard.ps1`	Start Streamlit.	`You can now view your Streamlit app`; open port 8501.
`$py -m pytest -m unit`	Fast tests, no services.	`73 passed, 1 skipped` (the skipped one needs Airflow's Python).
`$py -m pytest -m integration`	Tests against real PostgreSQL/Kafka/API.	`18 passed, 1 skipped`.
`.\scripts\run_tests_docker.ps1` / `.\scripts\run_tests_airflow.ps1`	Tests that need Spark / Airflow.	`5 passed` / `8 passed`.
`$py -m pytest -m e2e`	Whole pipeline, real services (about 3.5 minutes).	`4 passed, 1 skipped`.
`$py scripts\secret_scan.py`	Make sure no secret is committed.	`RESULT: clean`.

> **Shutting down safely**
>
> `.\scripts\stop_stream.ps1`, then `.\scripts\airflow_down.ps1`, then `docker compose stop`. Data stays in Docker volumes. **Never** delete `data/checkpoints` or the volumes unless you want to reprocess everything.

15. Reconciliation: proving the numbers add up

15.1 The idea

Reconciliation means "count the same thing at every stage and explain every difference". Anyone can print a count; a good engineer shows that **nothing was lost or invented**. `scripts/reconcile.py` works from the **source**: it recomputes what each stage *should* contain using the same rules, then compares with what is *actually* there. It exits with code 1 if a difference is unexplained. It never adjusts an expected number to make a row pass.

15.2 Three sections

1. **Run level:** each producer run versus the part of the file it covered (read, valid, rejected, published, dead-letter lines).
2. **Source level:** unique records over the *union* of all runs. Runs overlap, so run totals cannot simply be added.
3. **Pipeline stages:** Kafka → Spark → Parquet → PostgreSQL.

15.3 Real output (shortened)

`REAL OUTPUT · DOCS/EVIDENCE/07_POSTGRESQL/RECONCILIATION_PRODUCTION_FINAL.TXT`

```
== RUN LEVEL ==
[full-run-01] rows read                     99,441   99,441   +0  [OK] whole orders file
[full-run-01] valid                         99,427   99,427   +0  [OK] re-derived from that slice with the
same rules
[full-run-01] rejected                          14       14   +0  [OK] read - valid
[full-run-01] published = valid             99,427   99,427   +0  [OK] failed publishes: 0
[full-run-01] dead-letter lines = rejected      14       14   +0  [OK]
== SOURCE LEVEL ==
unique source rows covered                  99,441   99,441   +0  [OK] runs read 99,741 rows in total, so
300 rows were replayed more than once
unique rejected (distinct record_uid in DLQ)    14       14   +0  [OK] 16 dead-letter LINES exist because
overlapping runs rejected the same record again
== PIPELINE STAGES ==
4 messages in Kafka topic                   99,725   99,725   +0  [OK] 298 of them are repeats of an
already published record_uid
6 rows in Parquet (landing layer)           99,626   99,626   +0  [OK] consumed - invalid - in-batch
duplicates (99)
6b distinct record_uid in Parquet           99,427   99,427   +0  [OK]
7 rows in PostgreSQL core.orders            99,427   99,427   +0  [OK]
7b retry duplicates ignored by PostgreSQL      199      199   +0  [OK]
8b rows = distinct business_hash            99,427   99,427   +0  [OK] one row per natural key
RESULT: all differences are zero or explained
```

That file is the production state at the time of Phase 6. The final run (`docs/evidence/10_automated_tests/final_reconcile_production.txt`) covers six runs, after the re-ingestion and two Airflow runs, and ends with the same result line. The arithmetic of the first version:

> **Reading the pipeline arithmetic**
> 99,725 messages in Kafka − 99 repeats removed inside one batch − 0 invalid = **99,626** rows in Parquet. Distinct `record_uid` in Parquet = **99,427**; the other 199 Parquet rows are repeats that arrived in a later batch. PostgreSQL keeps one row per `record_uid` = **99,427**. Every number is explained.

15.4 The "14 versus 15" story

An early reconciliation said "15 rejected orders" while the correct figure from the source was 14. I did **not** change the expected number. I investigated (`scripts/explain_rejected_count.py`): the extra one was **one** `record_uid` (`92fc65a4...`) that two *overlapping* runs had both rejected, so it appeared as two dead-letter lines. The fix was to separate counting levels: run-level (lines) and source-level (distinct `record_uid`). Evidence: `docs/evidence/07_postgresql/rejected_14_vs_15_explained.txt`.

> **Why the tool itself is trustworthy**
> A separate test deliberately creates a mismatch and checks that `reconcile.py` returns exit code 1 (`reconciliation_mismatch_exit_code.txt`). A checker that can never fail proves nothing.

16. Errors, retries, checkpoints and recovery

A data pipeline is judged by what happens when something goes wrong. This chapter collects every protection in one place, then the real failures that were found and fixed.

16.1 The protections, layer by layer

Where	What can go wrong	Protection
Dataset	A file is missing or altered	SHA-256 fingerprint check (`check_dataset_fingerprint.py`; first Airflow tasks).
API	Network error, slow answer, HTTP 5xx	3 attempts with waits of 1 s then 2 s; cache of stored responses; the pipeline still works if the API fails (the order is kept, marked "not enriched"), and the failure is real, never faked.
Producer	A bad row	Eight rules; bad rows go to the dead-letter file with a reason code, good rows continue.
Producer → Kafka	Lost or doubled message on a retry	`acks=all`, idempotent producer, gzip; every failed publish is counted (`rows_publish_failed`).
Kafka → Spark	Spark stops or crashes	The **checkpoint** remembers the next offsets to read.
Spark batch	Malformed JSON, missing fields	Split valid/invalid inside every batch; invalid go to the stream dead-letter with code `MALFORMED_JSON` or `SCHEMA_MISMATCH`.
Spark → PostgreSQL	Same order arrives again	One transaction; `record_uid` is unique; an existing uid is counted as a retry duplicate and skipped.
Airflow	A task fails for a temporary reason	2 retries, 30 s apart; a failed task stops its followers.
Dashboard	A service is down	Real status lights; a short safe error message; no fake green.

16.2 Retry, at-least-once and the idempotent sink

> **Analogy: posting a parcel with tracking**
> If you are not sure the parcel arrived, you send it again. The receiver checks the tracking number: "I already have this one" and throws the duplicate away. Resending is safe because the receiver recognises repeats.

- **At-least-once delivery**: after a crash a message may be delivered *more than once*, never zero times.
- **Idempotent sink**: writing the same message again changes nothing. Here, the sink is PostgreSQL with a unique `record_uid`.
- Together this gives the *effect* of "each order counted once" in the warehouse. We never claim "exactly-once".

16.3 Checkpoint recovery: what exactly happens

For each micro-batch Spark writes `offsets/N` (the plan: "I will read from here to there") **before** processing and `commits/N` after it has finished. On restart:

- `offsets/N` and `commits/N` both exist: batch N is finished; the next batch starts after it.
- `offsets/N` exists but `commits/N` does not: batch N was interrupted; it is **run again** with the same offsets. Its rows may already have reached PostgreSQL, but that is harmless: they are retry duplicates.

REAL FILE · DATA/CHECKPOINTS/ORDERS/OFFSETS/0

```
{"ecommerce_orders":{"0":3338,"1":3345,"2":3315}}
```

Meaning: the first batch consumed 3,338 + 3,345 + 3,315 = 9,998 messages (partitions 0, 1, 2). Offsets are positions; the numbers are "up to, not including".

16.4 The recovery experiment (isolated checkpoint)

To prove recovery without risking the production checkpoint, an **isolated** experiment used its own checkpoint, Parquet folder and database tables (`04_spark/recovery_test.txt`, `07_postgresql/reconciliation_recovery_experiment_r2.txt`).

1. Start Spark on a backlog; let it commit batches 0 and 1; request a safe stop. **4,995 messages stay unread**.
2. While Spark is down, publish **2,000 new** orders and **99 repeats** of old ones.
3. Restart. The first batch starts at the committed offset (partition 0 at 1000), not at zero.
4. Result: Spark consumed 8,094 messages in total; Parquet 8,094 rows, **7,995 distinct**; PostgreSQL **7,995**; retry duplicates counted **99**; **18 of 18 checks passed**.

Meaning: nothing was lost (the unread backlog and the new orders were all processed), nothing was doubled in the warehouse (the 99 repeats were recognised), and no old data was reprocessed from scratch.

The failed first attempt (stopping *during* a batch) is kept as history: `04_spark/recovery_test_attempt1_stop_during_batch.txt`.

16.5 Proof that re-ingestion is harmless

On the real production data (99,427 orders) I published the first 200 orders again. Spark consumed **199** messages, inserted **0**, counted **199 retry duplicates**, and the count stayed **99,427** (`07_postgresql/reingestion_production.txt`). The two Airflow runs repeated the same proof with 295 messages each.

16.6 Dead-letter queue (DLQ) recap

A DLQ keeps the records you could not accept, **with the reason**, so they can be inspected and fixed instead of silently disappearing. Two places: `data/dead_letter/ingest/` (rejected by the producer: the 14 real orders) and `data/dead_letter/stream/` (rejected inside Spark: a malformed message). The real Spark path was exercised by publishing a deliberately broken message (`04_spark/malformed_message_test.txt`).

16.7 The honest failures list

These happened for real, were diagnosed, fixed and are kept as evidence. Telling them is a strength: it shows testing found real problems.

Failure	Cause	Fix
Spark would not start on Windows	Host Spark needs `winutils.exe`	Run Spark in Docker.
Spark image failed to build	Base image no longer had OpenJDK 17	Pin the `slim-bookworm` base.
Malformed JSON labelled `SCHEMA_MISMATCH`	`from_json` returns a row of nulls, not an error	Test JSON validity separately with `get_json_object`; the test was not changed.
Reference-table load crashed	numpy float not accepted by the database driver	Cast to plain `float` / `int`. Failed attempt kept.
First Airflow run failed	Permission denied on the root-owned checkpoint	Run the Airflow container as root (documented).
Dashboard: Kafka red while healthy	Unsupported client option	Removed it; added a test against the real broker.
Dashboard: money charted as text	Exact decimals not numeric to the chart library	Convert before charting.
Dashboard: internal docs printed	Streamlit prints bare expressions	Plain if/else + automatic source check.
Intermittent e2e failure	Stop request raced with a starting batch	New stop protocol (`StopRequested`); 3 stress runs and a final e2e run passed.
Fresh-environment test failed twice	"Filename too long" (very deep scratch folder)	Passed from a short path.
14 vs 15 rejected	Run overlap	Separated run-level and source-level counting.

17. Testing

17.1 Why several suites

The quick tests run in seconds with no services; the slow ones start the real system. Markers (`pytest -m ...`) choose which to run.

Suite	Needs / how it runs	What it covers	Final result
unit	Nothing external	Mapping, validators, producer ids and hashes, API client retry/cache, reconcile logic, stop classification, dashboard helpers, DAG structure, source checks	**73 passed, 1 skipped**
integration	PostgreSQL, Kafka, internet (real API)	Warehouse upsert and duplicate rules, dashboard queries, stored API results, one real API call	**18 passed, 1 skipped**
spark	Inside the Spark container	The Spark transforms: parse, validate, flatten, dedup	**5 passed**
airflow	Inside the Airflow container	DAG imports; 9 tasks; retries; no cycles; parameters	**8 passed**
e2e	Everything real, isolated checkpoint (3.5 min)	Full flow, malformed message, recovery after stop, idempotent reload	**4 passed, 1 skipped**
manual	A human	The four live demos	`PENDING_HUMAN`

Every run saved a timestamped log under `docs/evidence/10_automated_tests/`, with the git commit. Failed runs are kept (e.g. `pytest_e2e_20261016_FAILED_stop_race.txt`). The final summary: `final_verification_summary.txt`, which also shows: secret scan `clean`, every cited evidence path exists, production reconciliation `all differences are zero or explained`.

17.2 What is mocked and what is real

- **Unit tests** may use fake HTTP responses to check the retry *logic* (that is their purpose; they are labelled as such).
- **Integration and e2e tests use the real services**, including the real Frankfurter API. No test pretends to be Kafka or PostgreSQL.
- One test deliberately checks that the reconciliation tool **fails** on a real mismatch.
- A passing test is never presented as more than it is: unit tests are "tested", only saved real outputs are "verified with evidence".

17.3 Status vocabulary

`NOT_STARTED` → `IN_PROGRESS` / `BLOCKED` → `IMPLEMENTED_NOT_TESTED` → `TESTED` → `VERIFIED_WITH_EVIDENCE`. The last one is allowed only when an evidence file exists. See `docs/task_tracker.md` and `docs/evidence_register.md`.

18. Limitations (say them before you are asked)

- **Replay, not live.** The dataset is history (2016-2018). We replay it into Kafka; we never call these "live orders".
- **The API is not the order source.** It supplies the exchange rate. USD values are **estimates** (weekends use the previous business day: 24,112 orders).
- **At-least-once, not exactly-once**; deduplication happens in PostgreSQL.
- **Small infrastructure:** one Kafka broker (replication factor 1), one PostgreSQL, Spark in local mode in one container. No monitoring or alerting stack, no schema registry.
- **Parquet is append-only**, so it contains repeats; the warehouse is the deduplicated truth.
- **Dataset facts:** 775 orders have no items and 1 has no payment (kept and counted); timestamps have no time zone; the licence (CC BY-NC-SA 4.0) was read from a search excerpt and should be re-checked on the Kaggle page.
- **Airflow runs as root** in the container (demo trade-off). It orchestrates finite tasks only.
- **Retention:** Kafka forgets messages after 7 days; replaying again is safe.
- **Not done by the agent (human actions):** the four live demos and their screenshots, confirming the project title with the professor (dataset is e-commerce, not food delivery), asking whether a specific API was expected.
- **Future work:** more brokers, a schema registry, alerting, a live source, a cloud warehouse, row-level dashboard drill-down.

Full list: `docs/known_limitations.md`.

19. The four demonstrations

The professor expects you to perform four short demos live. They were **prepared and dry-run by the assistant** (real outputs in `docs/evidence/11_manual_demos/dry_runs/`), but the live run and its screenshots are : you must do them. Exact commands: `docs/demo_cheatsheet.md` and `docs/manual_demo_guide.md`. Demos 3 and 4 use an isolated demo database and checkpoint (`scripts\demo_setup.ps1`), so the production data is not touched.

Demo	What you do	What you should see and say
1. Dataset ingestion	Check the fingerprints, dry-run, publish 200 orders, look at one message and the partition counts.	`rows_read 200, valid 199, rejected 1`. "The dataset is real Olist data, joined from four files; the bad row is in the dead-letter file with its reason."
2. Real API	`api_demo_call.py --sample --no-cache`, then `--simulate-failure`.	`HTTP : 200`, 5 rows; then three attempts and a clear failure. "Frankfurter returns ECB rates, no key needed; retries wait 1 s and 2 s."
3. Data quality	Load a tiny file of 3 orders (one with an impossible value) twice.	2 valid + 1 rejected (`RANGE_ERROR`); after loading twice the warehouse still has 1 order per id; Parquet has 4 rows but 2 distinct. "Parquet keeps repeats, PostgreSQL does not."
4. Failure and recovery	Start Spark, stop it safely after 3 committed batches, publish more orders (and repeats) while it is down, restart.	Dry run: +1,000 new and +99 repeats; 3,995 unique orders; reconciliation exit code 0. "The checkpoint resumed from the right offset; nothing lost, nothing doubled."

> **Be honest during the demo**
> If something fails (for example no internet during Demo 2), show the real failure and explain it. Never edit an output. Fill in the "ACTUAL observed output" fields in the demo READMEs yourself.

20. Viva questions and answers

Format: **short answer** first, then the detail. Numbers are the measured ones.

Q1. What does StreamPulse do?

It replays the real Olist e-commerce orders (99,441 rows) into Kafka, processes them with Spark Structured Streaming into a Parquet landing layer and a deduplicated PostgreSQL warehouse, adds a BRL→USD value from the Frankfurter (ECB) API, orchestrates finite steps with Airflow and shows the results on a Streamlit dashboard.

Q2. Is this real-time?

The *processing* is stream-style (micro-batches every 5 seconds), but the *data* is historical, replayed. I call it "historical dataset replay", not live orders.

Q3. Why this dataset?

Olist is a real, public, multi-table Brazilian e-commerce dataset (CC BY-NC-SA 4.0) with orders, items, payments, reviews, products and dates: rich enough for joins, quality problems and business KPIs. It is e-commerce, not food delivery, which is why the title was changed and flagged for the professor.

Q4. What is the API and why that one?

Frankfurter: free, no key, ECB reference rates. The professor named no API; BRL→USD fits the data because it has payment values in reais and purchase dates. It is enrichment, not the order source.

Q5. Why Kafka? What are topic, partition, offset, key?

Kafka decouples producer and consumer and stores messages durably so Spark can stop and resume. A topic is a named log (`ecommerce_orders`); it is split into 3 partitions for parallelism; an offset is a message's position in a partition; the key decides the partition.

Q6. Why `order_id` as the key?

It spreads messages evenly (measured 33.4 / 33.4 / 33.2 %) and keeps all messages of one order in one partition. `customer_state` would put 66.9 % into one partition (a "hot" partition).

Q7. Micro-batches and triggers?

Spark Structured Streaming collects new messages into small batches. Trigger `5 seconds` runs a batch every 5 s (up to 10,000 messages); `availableNow` processes what is waiting and stops, which is how Airflow can run it.

Q8. What is a checkpoint and what happens after a crash?

Files recording which offsets each batch will read (`offsets/N`) and which batches finished (`commits/N`). After a restart Spark continues after the last committed batch and re-runs an interrupted one. Proven by the recovery experiment: 18/18 checks, no loss, 99 repeats recognised.

Q9. Parquet versus PostgreSQL?

Parquet: compressed columnar files, cheap, append-only landing layer (12 files, 100,120 rows incl. repeats). PostgreSQL: the structured warehouse with constraints and SQL, deduplicated (99,427 rows).

Q10. ETL or ELT?

Mostly ETL for the orders: they are validated and flattened in Spark before the warehouse. Raw payloads are kept in Parquet and the dead-letter files, and the analytics views are computed inside PostgreSQL (ELT-style).

Q11. How do you handle bad data?

Eight rules in `config/quality_rules.yaml`. Bad records are not dropped silently: they go to the dead-letter queue with a reason code (e.g. `INCONSISTENT`: 8 delivered-without-date and 6 canceled-with-delivery-date = 14 orders).

Q12. Explain deduplication and idempotency.

`record_uid` identifies one source row; `business_hash` identifies one order. The warehouse upsert runs in one transaction and classifies each record as new, retry duplicate (same uid) or source duplicate (same order, different uid). Invariant: attempted = inserted + retry + source. Re-ingesting 199 messages inserted 0.

Q13. Is it exactly-once?

No. It is **at-least-once delivery with an idempotent sink**, which gives the same *result* in the warehouse.

Q14. Why is the stream not an Airflow task?

A task must finish; a continuous stream never does. Airflow runs finite steps (including Spark with `availableNow`) with retries and logs. The real DAG run succeeded 9/9; a real failed run (permission denied) and a failure demo show retries.

Q15. How do you know the dashboard status lights are not fake?

Each light is a real check (a SQL query, Kafka metadata, file listing, HTTP call). When I really stopped Kafka and PostgreSQL the lights turned red and the KPI page showed a safe error.

Q16. Explain lineage and provenance.

Every message carries the source file's SHA-256, row number, run id and API retrieval time. The dashboard shows that all 99,427 warehouse rows come from one file fingerprint (`8df58ef3...`).

Q17. How do you know the numbers are right?

`reconcile.py` recomputes expected counts from the source and compares each layer; it exits 1 on an unexplained difference (tested on a deliberate mismatch). Result: all differences zero or explained.

Q18. What went wrong during the project?

Chapter 16.7: winutils, missing JDK, malformed JSON mislabelled, numpy float, Airflow permission, three dashboard bugs, a stop race, a path-length issue and the 14-vs-15 count. Each was diagnosed, fixed and kept as evidence.

Q19. What are the limitations?

Chapter 18: replay not live, estimated USD, single broker, at-least-once, Parquet repeats, Airflow as root, manual demos pending.

Q20. How would you scale it?

More partitions and brokers (replication factor 3), Spark on a cluster, a schema registry, monitoring and alerting, a managed warehouse, Airflow on Kubernetes.

Q21. What are the key numbers?

See Appendix B. Memorise: 99,441 read; 99,427 valid; 14 rejected; 3 partitions; 100,514 Kafka messages; 99,427 in PostgreSQL; 8.11 % late; 100 % enriched.

Appendix A: Glossary

Term	Meaning
Airflow / DAG / task	Workflow tool / a graph of tasks that cannot loop / one step.
API	A way for programs to ask another service for data over HTTP.
At-least-once	Every message is delivered one or more times.
availableNow	Spark trigger: process what is waiting, then finish.
Batch / micro-batch	A group of records processed together / a small batch made every few seconds.
business_hash	Fingerprint of one order (`order_id`); equal hashes = same order.
Checkpoint	Spark's saved progress (offsets and commits).
Consumer / producer	Program reading from / writing to Kafka.
Dead-letter queue	Store of rejected records with reasons.
Deduplication	Keeping one copy of repeated records.
Docker / container / volume	Packaged software / a running package / storage that survives restarts.
Enrichment	Adding information from another source (the USD value).
Envelope	The outer JSON: payload, raw, api_enrichment, source_category, provenance.
ETL / ELT	Extract-Transform-Load / Extract-Load-Transform.
Exactly-once	Each message processed exactly one time (not claimed here).
Idempotent	Repeating an action does not change the result again.
Join (LEFT JOIN)	Combining tables on a shared key; LEFT keeps every row of the left table.
Kafka / topic / partition / offset / key	Message log system / named stream / slice of a topic / position / value deciding the partition.
Landing layer	First stored copy of processed data (Parquet).
Lineage / provenance	Where data came from and how it travelled.
Parquet	Compressed columnar file format.
PostgreSQL / schema / view	Relational database / group of tables / saved query.
Reconciliation	Comparing counts across stages and explaining differences.
record_uid	sha256(file sha + ":" + row number), first 32 characters; identifies one source row.
Retry / backoff	Trying again / waiting longer between tries (1 s, 2 s).
Sink	The final destination of a pipeline (PostgreSQL).
Spark Structured Streaming	Spark's engine for processing data in micro-batches.
Streamlit	Python library for dashboards.
Upsert	Insert, or skip/update if it already exists.

Appendix B: Key numbers (all measured)

Fact	Value
Orders read / valid / rejected (distinct)	99,441 / 99,427 / 14 (8 delivered w/o date, 6 canceled with delivery date)
Kafka partitions / replication / spread	3 / 1 / 33.4, 33.4, 33.2 %
Messages in Kafka / Parquet rows / distinct / PostgreSQL	100,514 / 100,120 / 99,427 / 99,427
Reference tables	sellers 3,095; geolocation 19,010; products 32,951; reviews 99,224; items 112,650
Revenue BRL / USD estimate / average order	16,006,745 / 4,768,687 / 160.99
Delivered / late / late %	96,470 / 7,826 / 8.11 %
Average delivery time	12.56 days
Enrichment match rate / previous-business-day matches	100 % / 24,112
Orders without items / payments	775 / 1
Tests	unit 73, integration 18, Spark 5, Airflow 8, e2e 4 (all passed)
Airflow DAG	9 tasks, 2 retries, 30 s delay; real run success

Appendix C: Where to find the evidence

Folder in `docs/evidence/`	Contents
00_environment	Tool versions, Docker, ports
01_dataset	Profiling, fingerprints, join check, licence note
02_api	Real API calls, retry demo, cache
03_kafka	Topic, partition counts, key experiment, sample messages
04_spark	Batch logs, recovery experiments, stop stress runs, malformed-message test
05_data_quality	Rule experiments, dead-letter examples
06_parquet	Read-backs
07_postgresql	Schema, analytics views, quality checks, reconciliations, re-ingestion
08_airflow	DAG runs, failure demo, screenshots
09_dashboard	Seven page screenshots and the outage screenshots
10_automated_tests	Every test run, passed and failed, plus the final summary
11_manual_demos	Dry runs and the four demo READMEs (`PENDING_HUMAN`)

The master index is `docs/evidence_register.md`; the final audit is `docs/final_report.md`.